

1. ISO/IEC 12207:2010: Жизненный цикл ПО.

Группы процессов ЖЦ.

Жизненный цикл программы - это время, которое проходит от идеи до вывода программного обеспечения из эксплуатации.

Основные этапы ЖЦ ПО

1. Разработка требований
2. Анализ требований
3. Проектирование системы
4. Разработка
5. Тестирование
6. Внедрение
7. Эксплуатация
8. Вывод из эксплуатации

ISO/IEC 12207 - это стандарт, который детально определяет все что должна делать крупная компания для того чтобы ПО было разработано и что для этого требуется.

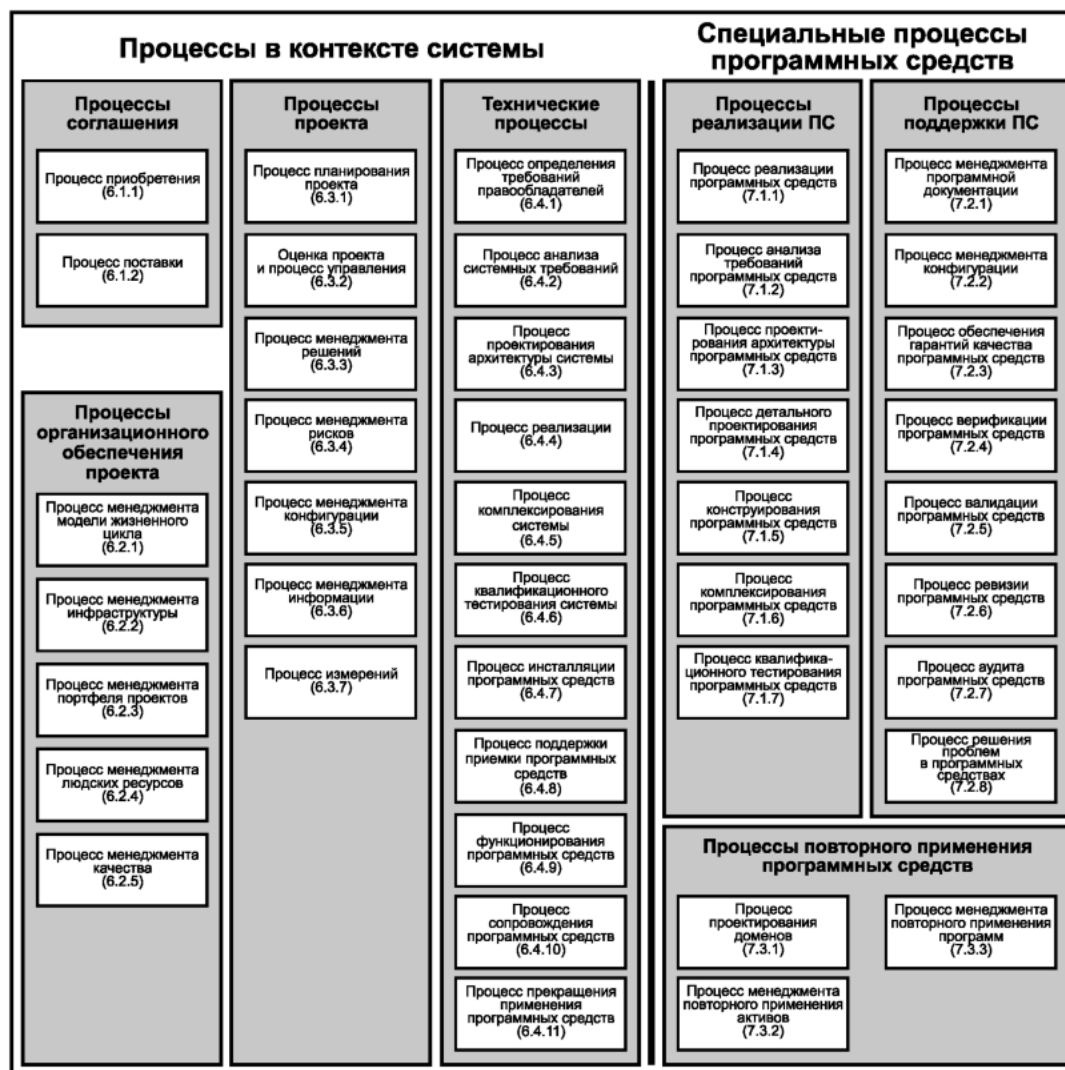
ЖЦ ПО состоит из большого количества процессов.

Каждый процесс в стандарте ISO описывается в виде:

Входные данные и ресурсы → Совокупность действий → Выходные данные и ресурсы

Группы процессов ЖЦ (в скобках кол-во процессов в группе)

1. Процессы в контексте системы
 - a. Процессы соглашения (2)
 - b. Процессы организационного обеспечения проекта (5)
 - c. Процессы проекта (7)
 - d. Технические процессы (11)
2. Специальные процессы программных средств
 - a. Процессы реализации ПС (7)
 - b. Процессы поддержки программных средств (8)
 - c. Процессы повторного применения программных средств (3)



2. Модели ЖЦ (последовательная, инкрементная, эволюционная).

Модель Жизненного цикла ПО - это структура, определяющая последовательность выполнения и взаимосвязи процессов, действий и задач на протяжении всего ЖЦ

1. Последовательная

Все требования уже определены. Разработка происходит в один этап, который разбивается по людям.

Подходит для разработки уникальных продуктов (Например марсоход, требования для которого определены заранее, и который не будет переиспользоваться). Но, часто невозможно продумать все требования

заранее, что может привести к появлению артефактов в конце. Из плюсов, можно достаточно точно рассчитать сроки и стоимость

2. Инкрементная

Все требования определены, но есть возможность разбить работу в несколько этапов. Между этапами возможны коррекции, но это по прежнему не гарантирует, что все требования будут учтены вовремя

3. Эволюционная

Требования целиком не определены, и доопределяются в процессе разработки. Но это приводит к тому, что ПО может накапливать в себе дефекты, из-за чего может понадобится рефакторинг.

Используется в современных методологиях разработки (например SCRUM), но чаще скорее встречается инкрементно-эволюционная

4. Модель формальных преобразований

Программа описывается в виде модели, и потом, при помощи формальных преобразований, превращаете одни модели в другие, и таким образом получается программное обеспечение

3. Водопадная (каскадная) модель.

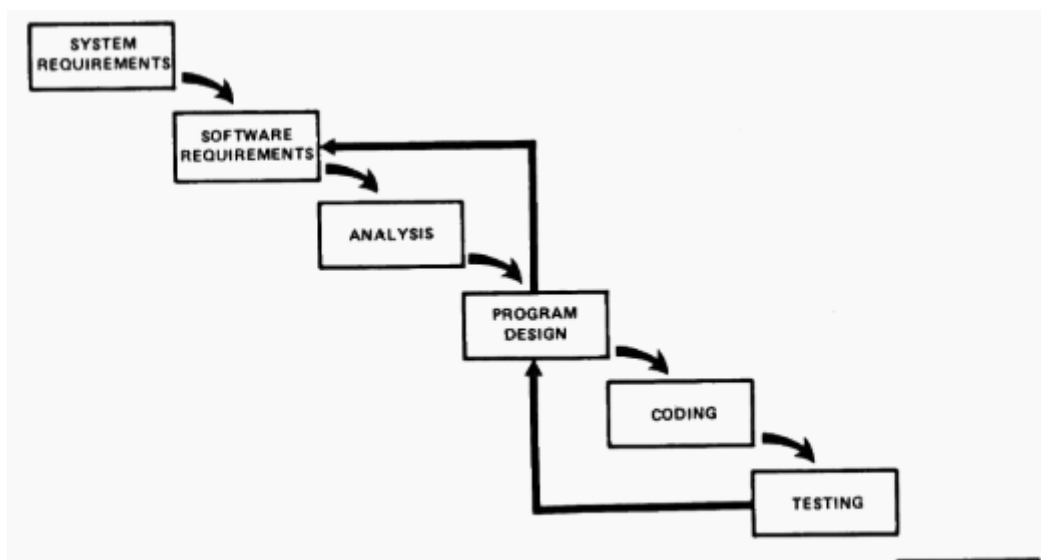
В начале у нас есть требования к системе, по сути грубые функциональные требования, которые система должна выполнять

Дальше мы из системных требований переходим к требованиям частей ПО, модулям

После этого отдельная команда анализирует полученные требования

В начале выполняется стадия общего архитектурного дизайна программы

Далее мы переходим к кодированию, тестированию и соответственно к использованию системы в продуктивной работе



Но если наша система реализует что-то не то, то мы это обнаружим только на стадии тестирования. И большая часть работы уже была сделана перед тестированием, и приходится откатываться назад либо на архитектурный дизайн, либо до требований если всё совсем плохо, и переделывать большой кусок работы

Такая модель подходит для небольших программ

4. Методология Ройса

Методология Ройса основана на водопадной модели, но Ройс изменил её:

1. Предварительный дизайн

Помогает дизайнерам убедиться в том, что требуемые характеристики могут быть реализованы.

В нём предлагается спроектировать, определить и создать модели обработки данных, даже если они будут требовать переделок и сделать Doc2 - обзор будущей системы

2. Документирование дизайна

(все документы представлены на картинке)

3. Do it twice

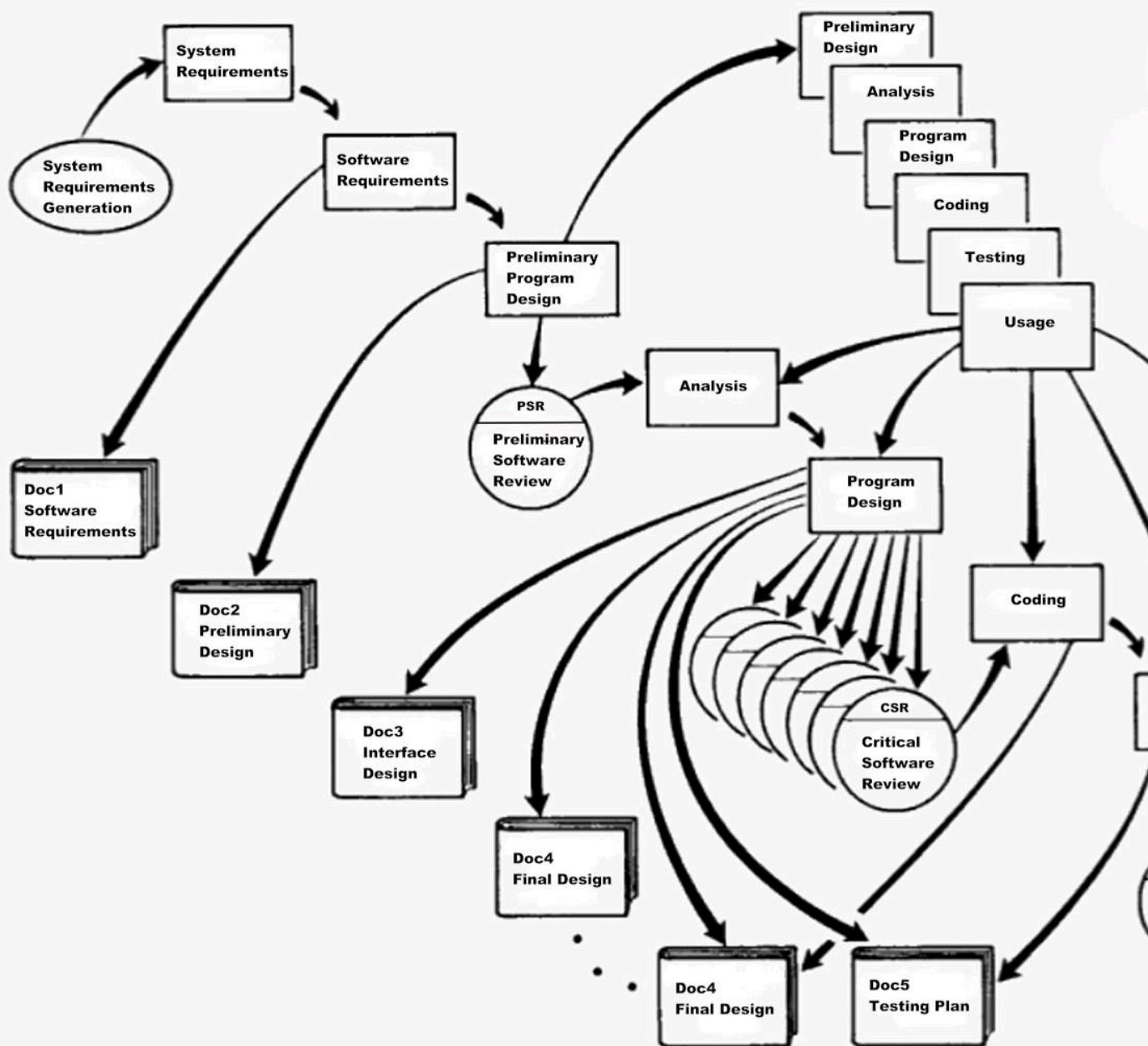
Предлагается провести тестовую разработку, параллельную основному процессу. Это позволяет подтвердить или опровергнуть основные характеристики ПО

4. Планирование, контроль и мониторинг тестирования

При планировании тестирования предлагается исключить дизайнера из процесса, провести “визуальную инспекцию” - повторный просмотр кода другим лицом, что поможет устранить визуально заметные дефекты, протестировать каждый логический путь программы и после исправления простых ошибок провести проверку в необходимом тестовом окружении

5. Подключение пользователя.

Ройс предложил подключить пользователя на ранних этапах перед постановкой продукта (Предварительный, критический и финальный просмотр) (круги на картинке - PSR, CSR, FSAR)



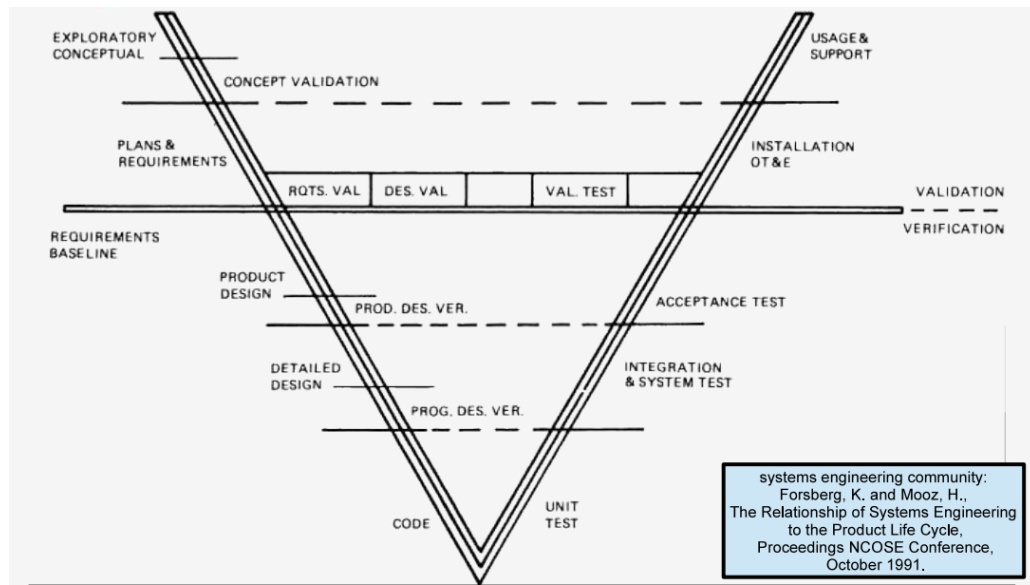
Советую выучить. Хотя и страшно выглядит, тут на самом деле всё просто

5. Традиционная V-chart model

В основе этой модели лежит та же последовательность шагов, что и в водопадной модели, но каждому уровню разработки соответствует свой уровень тестирования.

Для проведения тестирования сначала необходимо определить корректное поведение программы, так как целью является определение соответствия поведения программы эталонному корректному поведению

Динамическое тестирование включает компьютерное исполнение тестов (правая сторона V-модели), а статическое тестирование предполагает проверку артефактов разработки без их компьютерного исполнения, что помогает выявить грубые ошибки на ранних этапах разработки



6. Многопроходная модель (Incremental model).

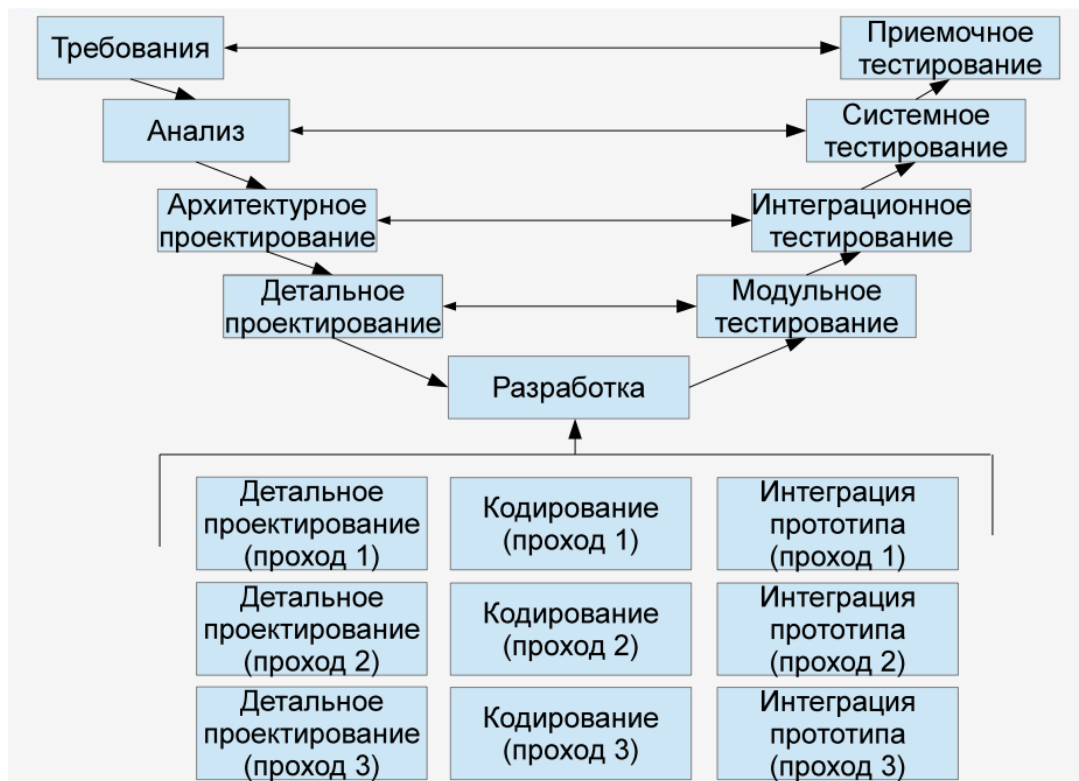
Из-за сложности разработки продукта в один этап, было предложено разбить продукт по отдельным функциональным и техническим требованиям, а в последствии проектировать, реализовывать и интегрировать воедино в несколько проходов разработки в виде отдельных сборок, которые называются инкрементами функционала. (или просто инкрементами)

Многопроходная модель представляет собой ту же V-образную модель, но разработка и интеграция происходит в несколько этапов. Это позволяет снизить стоимость изменений требований заказчика, и ему становится легко контролировать и вносить правки в функционал. Более того заказчик может использовать в своей работе частично разработанную систему. Современные гибкие (Agile) методологии опираются именно на этот подход.

Но из-за устаревания и деградации со временем система начинает требовать рефакторинга, что стоит денег и трудно для заказчика.

Большие проекты, разработка и поддержка которых осуществляется параллельно разными командами, требуют стабильную неизменяемую архитектуру и API, которые необходимо спланировать заранее. Также трудно управлять проектом, т.к. документы по программным сборкам сложно поддерживать из-за большой скорости изменений

Ещё сложным является заключение контрактов на разработку, т.к. обычно предполагаются фиксированные суммы оплаты на разработку этапов, где сложно учесть потенциальное появление изменений, и включить почасовую оплату.



7. Модель прототипирования (80-е).

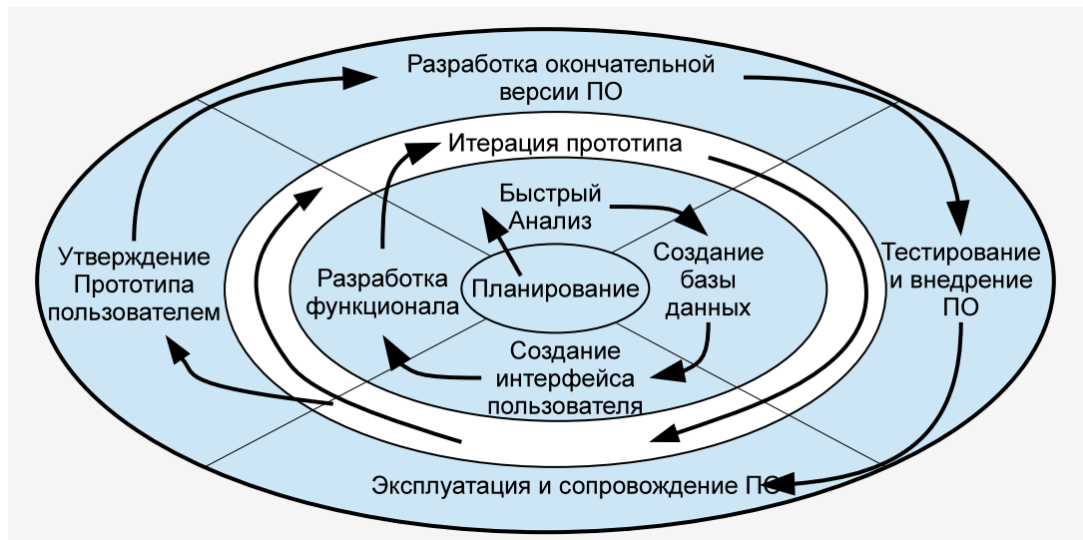
Существует проблема непонимания, и бывали случаи, когда писали ПО, которое вовсе не нужно.

И так же с приходом технологий в бизнес, нельзя было тратить много времени на разработку полного продукта. Нельзя было сначала всё спроектировать, и принести готовый проект через 2 года. За это время можно было проиграть всю конкурентную борьбу.

Модель прототипирования нацелена на быстрое создание прототипов, чтобы пользователь мог оценить, то ли это что ему нужно. И, если прототип не подошёл пользователю, создаётся следующий прототип, который уточняет архитектуру в рамках функциональных требований. И

так до тех пор, пока пользователь не оценит прототип. После этого уже начинается разработка основной версии продукта.

В современных методах для разработку применяются развитые средства макетирования, предназначенные для создания прототипов. Это так называемые “мокапы” (mock-up), простые визуальные модели пользовательского интерфейса, которые можно дать понажимать пользователю для оценки удобства



8. RAD методология.

RAD - rapid application development.

Идея в том, чтобы вовлечь в разработку пользователей, и чтобы они сами решали, чего они хотят (как бы разрабатывали сами). По сути приложение строится из отдельных частей (стандартных абстрактных блоков), а пользователь уточняет эти части (как бы тыкая галочки). И это оказалось удобным для бизнеса. Но для работы с RAD моделями, как программист, нужно знать ещё и область для которой это всё разрабатывается.

Предпосылкой появления данной методологии была потребность в автоматизации бизнеса крупных компаний. До этого подобной автоматизации практически не существовало. Все программы писались в текстовых редакторах, для подключения к бд формировались отдельные запросы, и невозможно было разработать ПО в терминах бизнес-логики. Эта методология позволила бизнесу самому проектировать бизнес-функции ПО

Для разработки RAD часто используются CASE системы.

CASE - системы, это системы, которые полностью покрывают ЖЦ разработки ПО, начиная от требований до отслеживания ошибок внутри ПО. В

Case-средствах производится автоматизированный учёт и анализ требований, построение моделей и генерация первоначального кода на основании этих моделей. Так же в них интегрированы модули для работы с системами версий, проведения тестирования, управления дефектами, потоком задач для разработчиков и тп.

9. Спиральная модель.

Боем перенёс работу с рисками из финансовой сферы на разработку ПО, оставив своё тестирование, разработку прототипов и тп.

В спиральной модели, у нас прототипы “раскручиваются”:

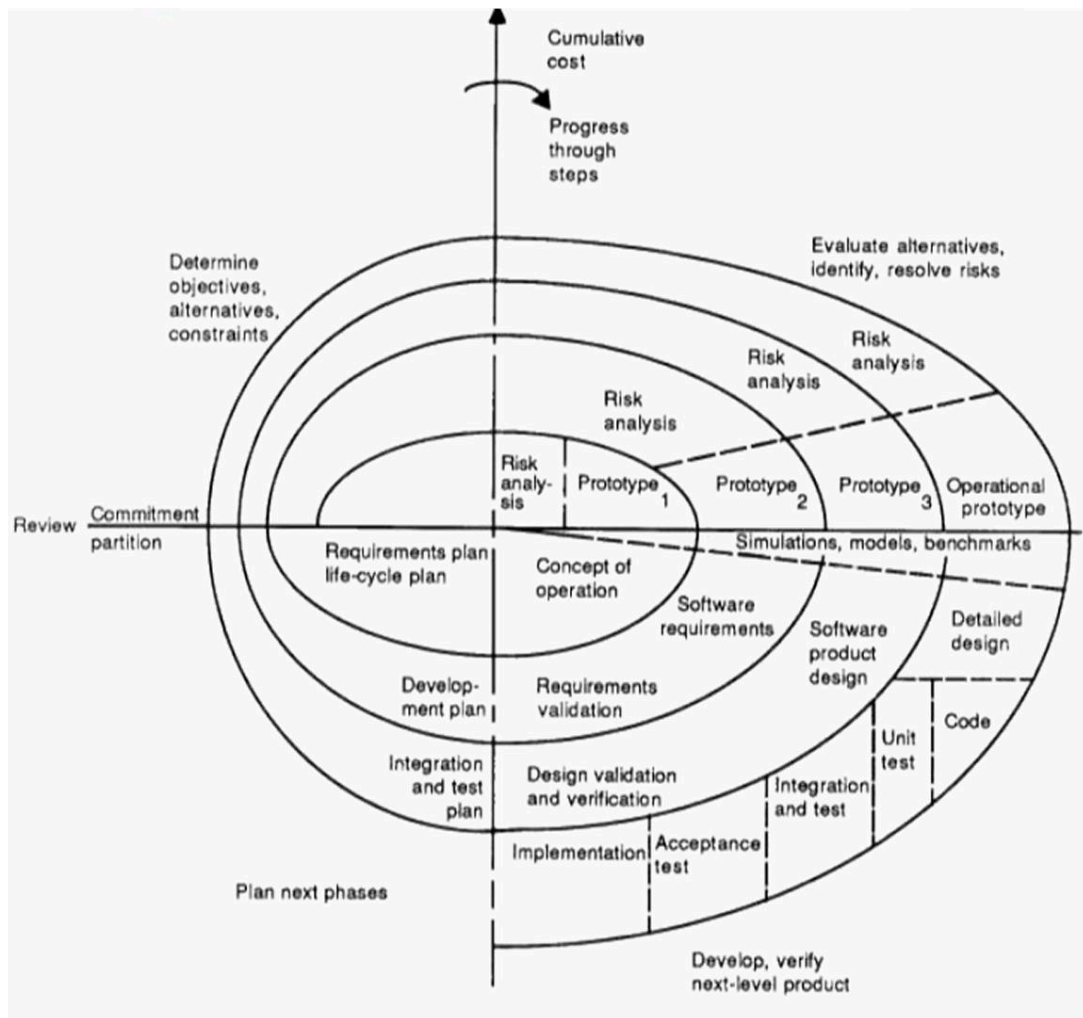
Каждый виток спирали представляет собой одну фазу разработки продукта, которая предназначена для разработки очередной версии ПО или прототипа.

1. Сначала происходит постановка целей для каждой итерации
2. Затем анализ рисков (за счёт снижения рисков минимизируется число действий, необходимых для разработки ПО)

Сверка с картой рисков (на сколько вероятен риск на след итерации, при этом анализируются все виды рисков: технические, политические, ресурсные и тп)

3. Разработка и валидация полученной версии.
4. В конце витка производится планирование следующего
5. В конце последней итерации повторяется часть тестирования и V образной модели

Заслуга Бозма в том, что включил изменения как неотъемлемую часть процесса разработки. И определил, что эти изменения могут быть приняты, отклонены или проигнорированы, в зависимости от рисков, которые они несут



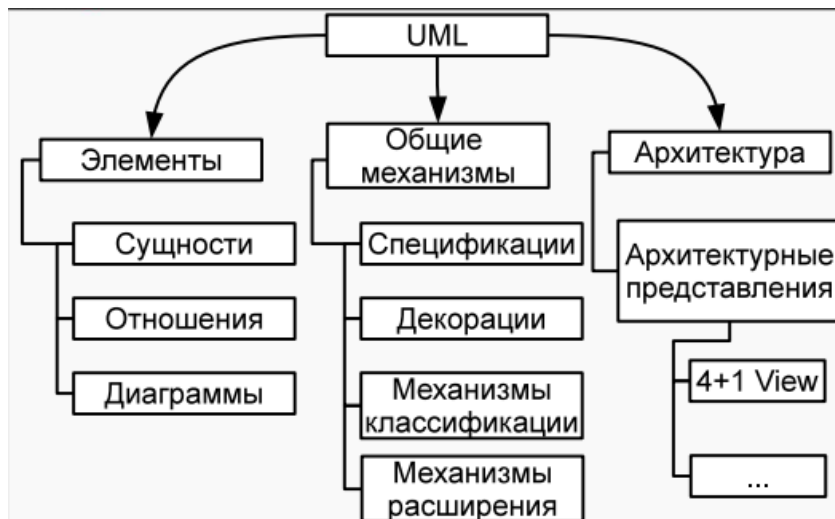
10. UML Диаграммы: Структурные и поведенческие.

Примеры диаграмм.

https://youtu.be/WQjJZyV4eNw?si=TGOsq_hmDnZwCwm5&t=2455 (стр 30) - структурные

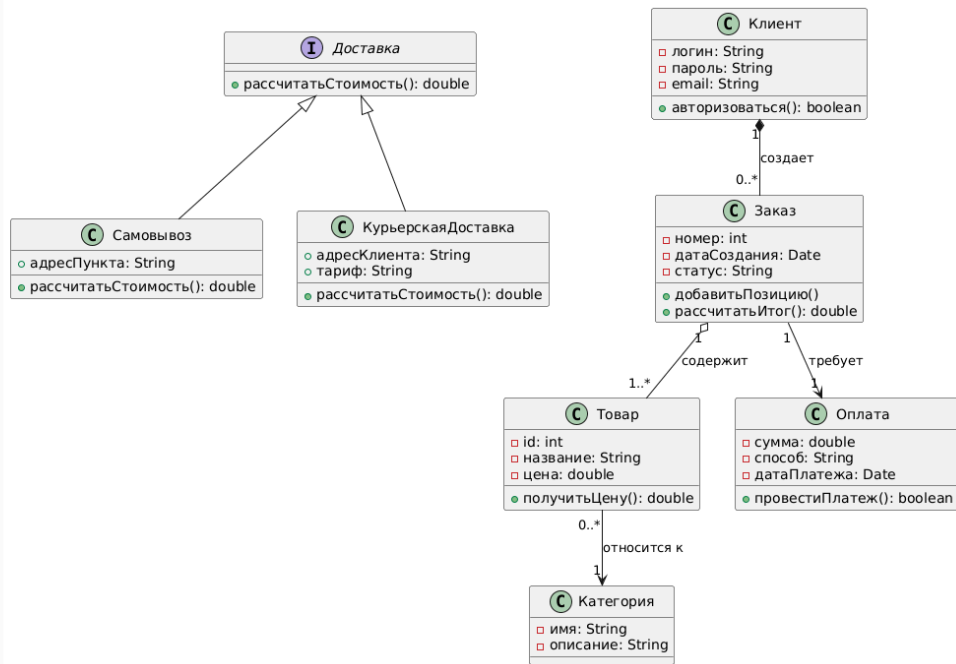
<https://youtu.be/WQjJZyV4eNw?si=TCcxlp64m5Xk6rhz&t=2870> (стр 31) - поведенческие

Язык UML – это графический язык моделирования общего назначения, предназначенный для спецификации, визуализации, проектирования и документирования всех артефактов, создаваемых при разработке программных систем.



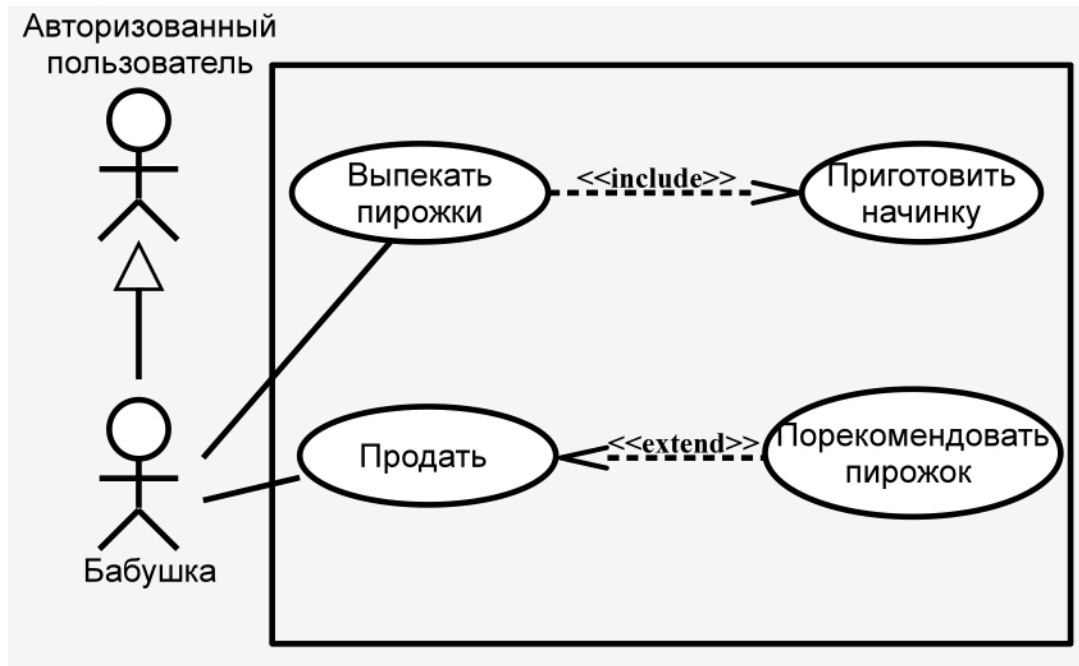
Структурные:

- **Объект** – сущность, обладающая уникальностью и инкапсулирующая в себе состояние и поведение.
- **Класс** – описание множества объектов с общими атрибутами и операциями. Понятие класса совпадает с принятым в ооп.
- **Интерфейс** – множество операций, определяющее набор услуг, которые могут быть запрошены потребителем и предоставлены поставщиком услуг.
- **Кооперация** (collaboration) – совокупность взаимодействующих объектов (на кухне работает несколько бабушек, которые делают пирожки).
- **actor** – внешняя, по отношению к системе сущность, являющаяся инициатором взаимодействия с системой. Действующее лицо может быть представлено ролью, другой системой или временем.
- **Компонент** – модульная часть системы с четко определенным набором требуемых и предоставляемых интерфейсов.
- **Артефакт** – результат работы, единица осмысленных действий в результате разработки, (как в RUP)
- **Узел** (node) – вычислительный ресурс, система или комплекс, на котором размещаются и используются компоненты и артефакты.



Поведенческие:

- **Прецедент** (use case) — сущность, которая определяет акт использования системы внешним действующим лицом, которая производит для него важный результат. Что система должна сделать. (заказа пирожков и рекомендации пирожков)
- **Состояние** — период в общем времени жизни объекта, в котором характеризуется уникальным и семантически различным набором значимых внутренних атрибутов объекта. Состояние может быть также выражено продолжительным монотонным действием.
- **Деятельность** — объединенное в логически значимое для предметной области множество (последовательность) действий. **Действие** в этом случае является простейшим вычислением.

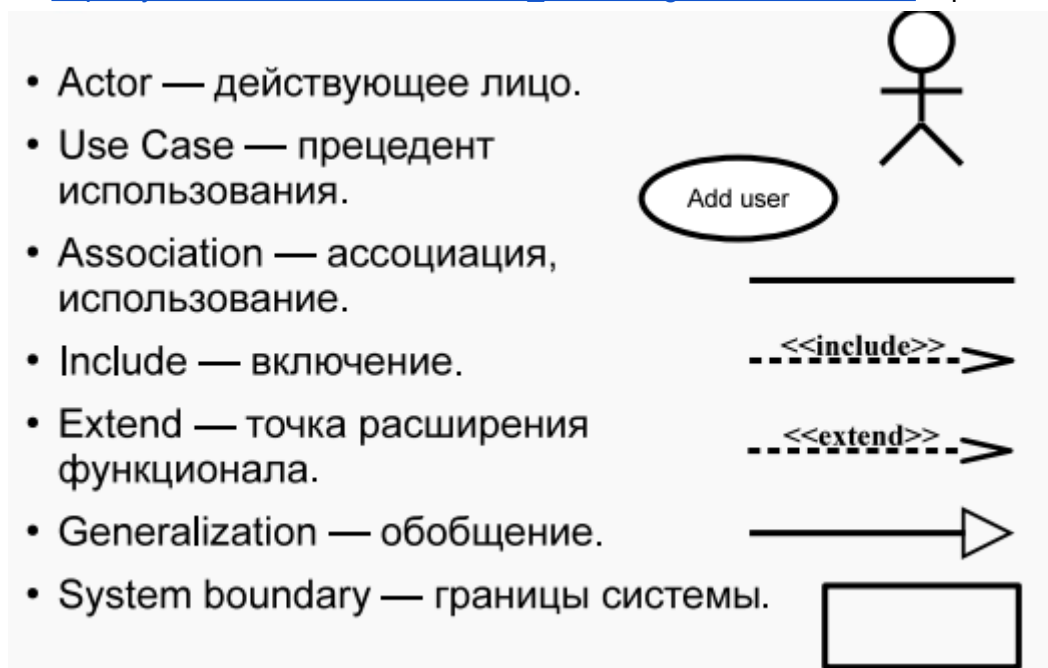


Дополнительные сущности представлены **пакетом** — логической группировкой сущностей. Никакого другого смыслового наполнения кроме группировки пакет не предоставляет. Для облегчения читаемости диаграмм предназначены **комментирующие сущности**.

11. UML: Use-case модель. Элементы диаграммы, пример диаграммы.

(это из 3 лекции)

https://youtu.be/Lnd-BW7IOBQ?si=2_vmwuSGg48Bm6nh&t=3868 стр 54



Главным элементом модели является действующее лицо или **актор**. Экторы никогда не являются частью системы. Время также может являться эктором.

Внутри прецедента использования пишется глагол или глагольная фраза, которая указывает, на то, что именно должна сделать система (пример: "Пользователь вводит логин и пароль").

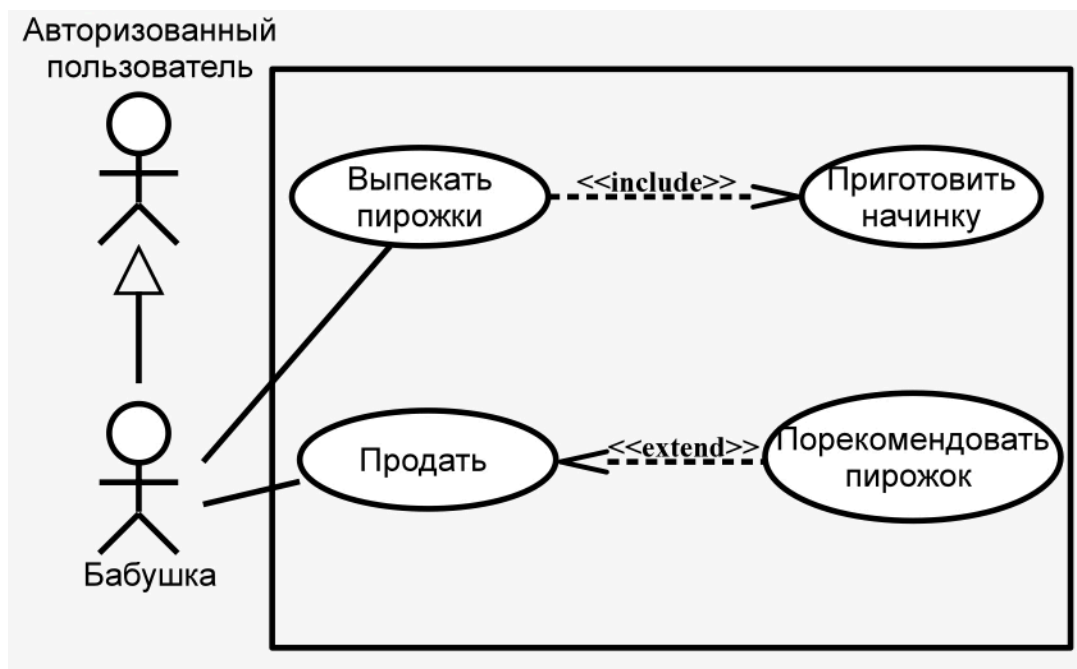
Association (ассоциация) — на диаграмме прецедентов связь, указывающая на то, что эктор использует систему и выполняет указанный Use Case.

Стереотип **Include** (включение) организует иерархию прецедентов использования системы и позволяет включать одну общую деятельность в несколько Use Case'ов.

Стереотип **Extend** (точка расширения функционала) указывает расширение от базового функционала.

Связь **Generalization** (обобщение) — это так называемая связь "is a", показывающая, что данный объект является потомком более высокого класса объектов. Например, класс "утка" может являться потомком класса "птица".

Элемент **System boundary** (границы системы) важен тогда, когда система разбита на подсистемы, и необходимо разграничить их содержимое.



Use-case модель отражает требования к программному продукту и представляет из себя диаграмму вариантов использования программного продукта.

Главным элементом модели является действующее лицо (actor) — пользователь, который непосредственно взаимодействует с системой. Под пользователем указывается его роль. Эктор никогда не является частью системы. Под эктором тоже можно считать время (в ряде случаев)

Прецедент содержит в себе действие, выполняемое в системе (например, “Пользователь вводит логин и пароль”. Прецеденты одних модулей могут выступать акторами для других модулей.

Ассоциация используется в диаграмме прецедентов для указания того, что актор использует систему и выполняет указанный прецедент.

Также в Use-Case модели используются следующие стереотипы:

- **Стереотип Include (включение)** организует иерархию прецедентов использования системы и позволяет включать одну общую деятельность в несколько Use-Case'ов.
- **Стереотип Extend (точка расширения функциональности)** указывает, собственно, на точку расширения функциональности. Например, при просрочке возврата книги в библиотеке с пользователя взимают штраф. То есть при наличии определенных условий будет применена дополнительная функциональность. Не стоит путать со стереотипом Generalization.

Связь Generalization (обобщение) указывает на отношение is a (А является В), показывающая, что данный объект является потомком более высокого класса объектов.

Напоследок, в Use-Case диаграмме используется элемент System Boundary (граница системы). Он представляет интерес, когда есть несколько подсистем, и требуется знать разделение между ними.

12. UML: Диаграмма классов. Элементы диаграммы, пример диаграммы.

<https://youtu.be/WQjJZyV4eNw?si=Je8ujRlvhZidNuE3&t=3468> (стр 33)

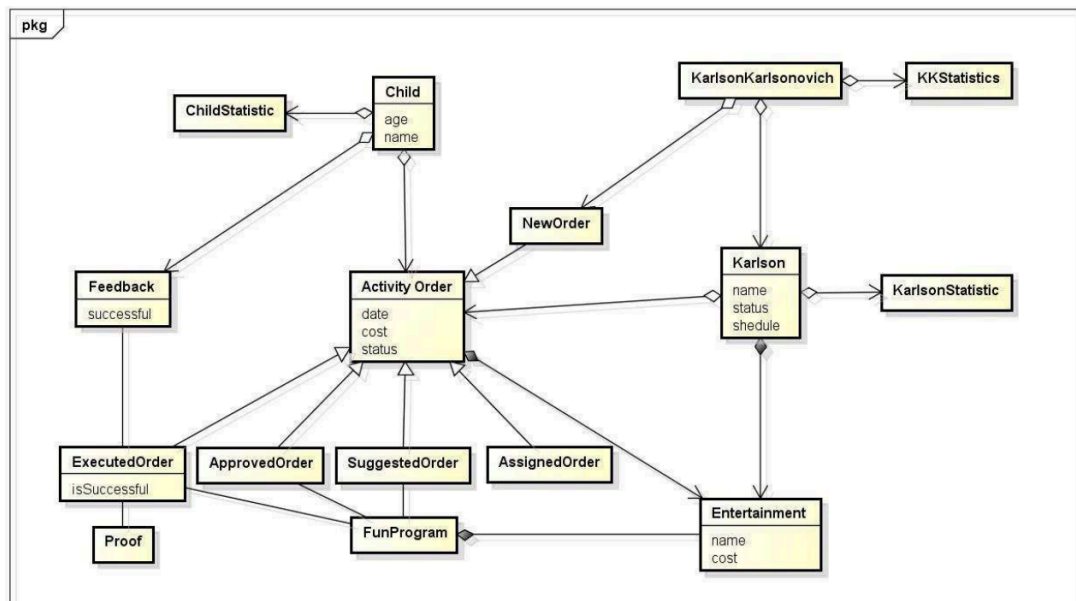


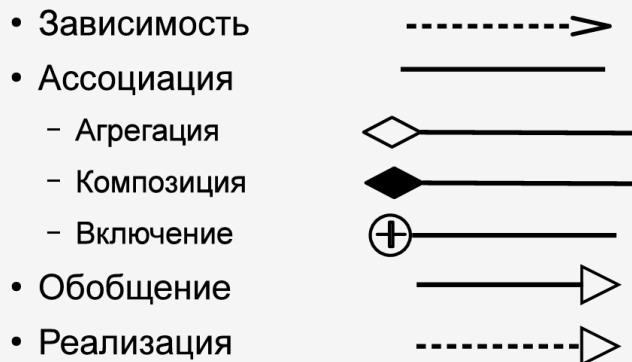
Диаграмма классов (она же **доменная модель**) является примером структурной диаграммы и отображает в первом приближении реализованную предметную область. Преимущество такой диаграммы заключается в ее наглядности.

Стереотипами для диаграммы классов могут быть:

- Interface
- abstract - пометка абстрактного класса
- enumeration - перечисляемый тип (enum)

(Сразу хотелось бы предупредить — данная модель взята из студенческих работ и в ней есть некорректное использование элементов языка UML, которые вам необходимо будет найти самостоятельно.) надо будет тоже сделать свой норм пример)

Основным преимуществом данной модели, даже несмотря на ее явные недостатки, является наглядное изображение предметной области. Данная диаграмма по мере дальнейшей работы на стадиях проектирования может быть уточнена и расширена. Например, могут быть добавлены типы данных и область их видимости, кратность и роли ассоциаций и т.д.



Отношение **зависимости** используется, когда необходимо показать, что исходный элемент зависит от целевого элемента и изменение целевого элемента может повлиять на исходный.

Ассоциация показывает, что один элемент связан с другим.

- **агрегация**, где целевой элемент является частью исходного элемента, и может существовать как отдельный элемент. Так связаны, например, автомобиль и колеса;
- **композиция** — строгая, форма агрегирования, при которой часть не может существовать без целого, и доступна только через точки взаимодействия, представленные целым. В качестве примера можно привести автомобиль и двигатель.
- **включение** — исходный элемент содержит целевой элемент, и имеет доступ к его пространству имен; Обычно данная ассоциация используется в диаграмме пакетов.

Обобщение — исходный элемент является специализацией более обобщенного целевого элемента и может замещать его. Этим отношением выражается в ооп наследование. Стрелка направлена к классу-родителю.

Реализация — исходный элемент гарантированно выполняет контракт, определенный целевым элементом, т. е. реализует его интерфейс.

Специфика всех отношений может уточняться при помощи стереотипов.

Различные диаграммы для этого определяют стандартные стереотипы.

Например зависимость в диаграмме пакетов может быть уточнена пятью стереотипами: use, import, access, trace, merge.

13. UML: Диаграмма состояний. Элементы диаграммы, пример диаграммы.

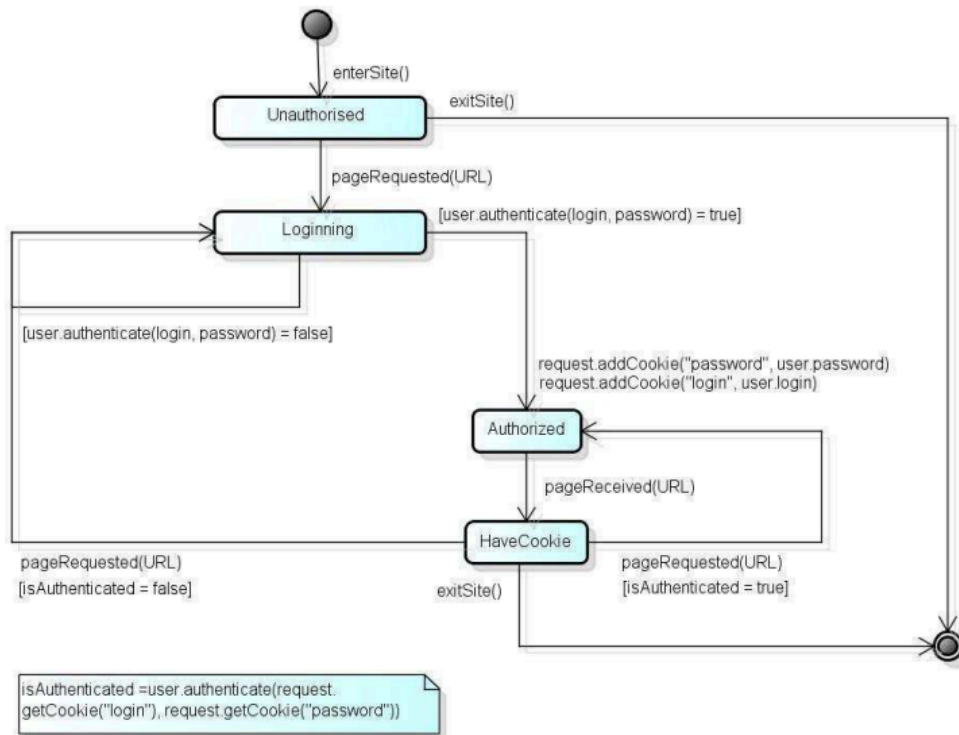


Диаграмма состояний (диаграмма конечных автоматов) является примером поведенческой диаграммы и отображает модель “Finite state machine” (программа конечных автоматов), представляющая собой набор состояний и условием переходов между ними.
Детальнее, чем доменная.

Описывает предметную область на уровне кода, при этом позволяет это сделать без перегрузки диаграммы

(по его словам пример тоже некорректный)) из-за перепутанной авторизации и верификации

14. UML: Диаграмма размещения. Элементы диаграммы, пример диаграммы.

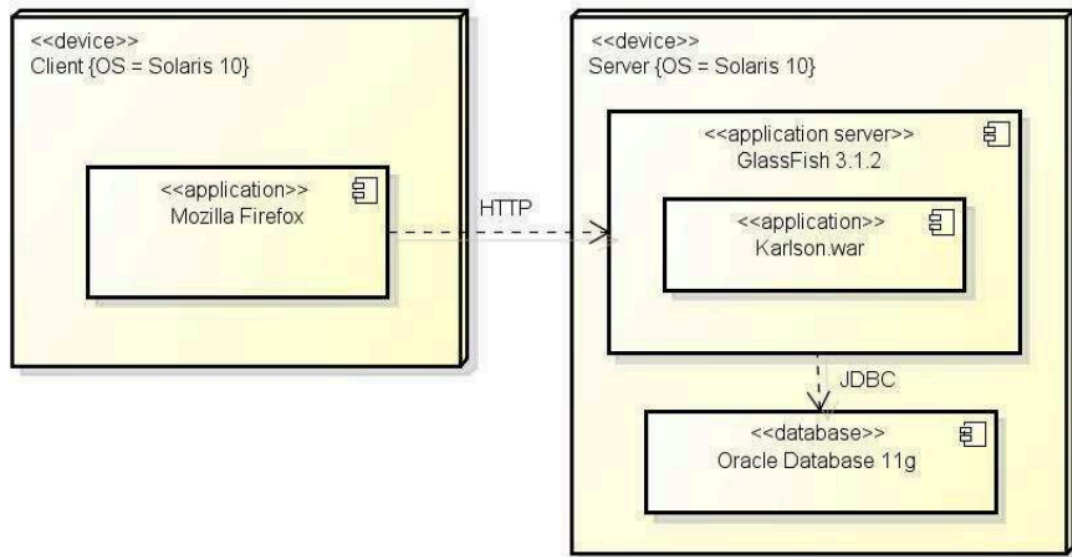


Диаграмма размещения является примером структурной диаграммы и отображает физическую архитектуру размещения программного обеспечения, а также возможности его использования. Например, на этой диаграмме может быть нарисована серверная машина, в которой имеется сервер приложений Java, в нем крутится наша лаба по веб-программированию, а также может иметься пользовательская машина с запущенным в ней браузером, и соединение происходит по протоколу связи HTTPS.

15. *UP методологии (90-е). RUP: основы процесса.

<https://youtu.be/WQjJZyV4eNw?si=KbhTYX0YItYednUy&t=40> (20стр)

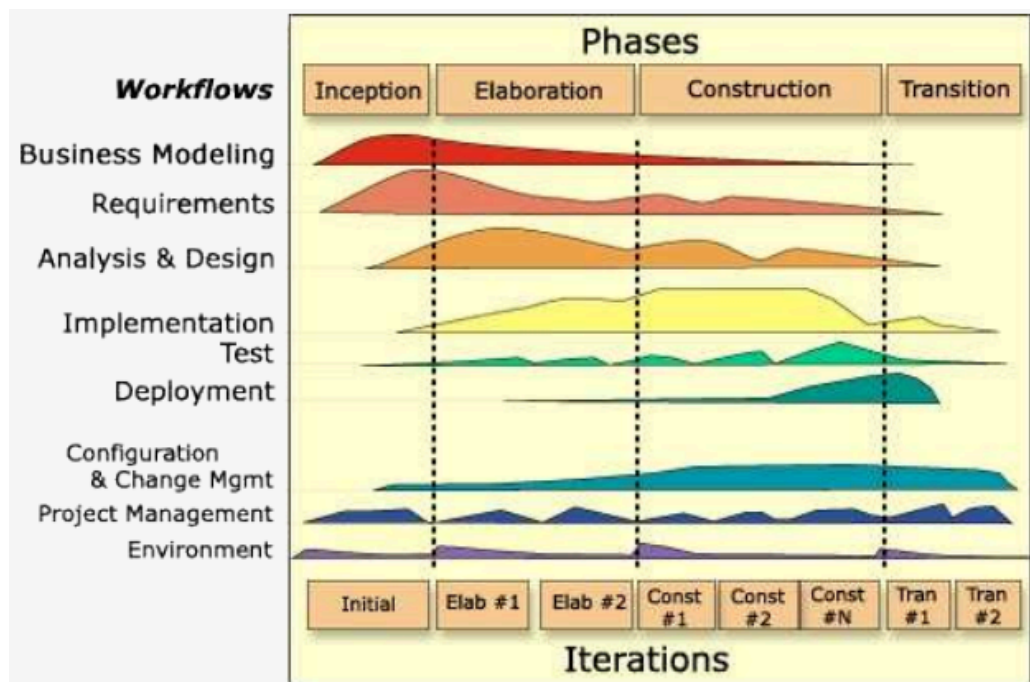
В то время стала массовой и широко известной объектно-ориентированная парадигма + появилось много методик для создания программ с учетом объектно-ориентированного подхода. Собрав всё это воедино, создали унифицированный процесс разработки, вобравший в себя все известные тогда характеристики других методов разработки.

Это процесс представлял всю разработку в виде инкрементально-эволюционного процесса. Весь процесс разработки разбит на фазы. Помимо них, в RUP также существует понятие дисциплины.

Дисциплины представляют собой набор правил и указаний, которые необходимы для решения определенной задачи, например, определения

требований или тестирования. Все дисциплины предназначены для того, чтобы организовать разработчиков, дать им характерные подходы, чтобы каждая определенная роль процесса разработки могла выполнять необходимые и требуемые для нее действия.

Всего фазы четыре: **Inception** (фаза начала), **Elaboration** (фаза проектирования), **Construction** (фаза создания продукта), и **Transition** (фаза внедрения продукта на стороне заказчика).



Например, анализируя график деятельности на слайде, можно увидеть, что работы по дисциплине Deployment активнее всего ведутся в фазах Construction и Transition, так как в это время уже существует продукт, который можно разворачивать. До этого по данной дисциплине проводилось, в основном, планирование. В дисциплине Implementation, который формирует подходы для эффективной разработки ПО, есть всплеск в фазе Transition, так как там производятся исправления продукта после размещения на стороне пользователя.

В RUP чётко определено то, как строится весь процесс. В процессе в явном виде описаны все **роли**, тем или иным образом связанные с созданием ПО. Всего в RUP входят около 30 различных ролей.

Роль — группа обязанностей, которую берет на себя участвующий в разработке для выполнения своей повседневной деятельности.

Например, в роль программиста входят создание кода, фиксация изменений в системе контроля версий, разработка модульных тестов для кода, поиск справочной информации для решения возникающих проблем, получение заданий в соответствующей системе и фиксация затраченного на их выполнение времени.

Таким образом, у программиста есть свой бизнес-процесс, и деятельность, входящая в этот процесс, составляющая роль программиста.

У каждой роли, участвующей в разработке, есть свой набор деятельности.

На входе и выходе деятельности используются, создаются и модифицируются **артефакты**. **Артефакт** в разработке — это любой результат труда роли, например, проектный документ, графическое изображение, часть кода, исполняемый файл, и т. п. Артефакты создаются на основании **инструкции** (guidelines) и шаблонов, с учетом контрольных точек и других элементов процесса.

Деятельность, которая производится ролью, всегда производится согласно установленным правилам и при помощи принятых в компании средств. Компании могут изменять процессы для того, чтобы они больше подходили под текущую структуру компании, RUP предлагает процессы только в качестве основы.

В отличие от других моделей ЖЦ и методов, в RUP каждый элемент процесса детально описан и связан с другими. Сам по себе RUP как программный продукт представляет набор связанных гиперссылками страниц HTML, каждая из которых описывает элемент процесса разработки.

Любая фаза в RUP заканчивается вехами. **Веха** — это момент времени для принятия решения о дальнейших действиях: переход на следующую фазу, либо проведение каких-либо дополнительных работ на данной фазе. Решение принимается заинтересованными сторонами (stakeholders).

Стейкхолдеры или заинтересованные стороны — лица, материально заинтересованные в создании ПО, такие, например, как инвесторы, руководство высшего звена заказчика и исполнители работ. Они хотят разработать программный продукт и получить с этого прибыль в той или иной форме. Обычные пользователи не являются заинтересованными лицами, так как для них ПО — инструмент работы, а не источник дохода.

Просто для общего понимания неплохо понимать Дух RUP(хотя в билетах нет этого):

- *Атаковать риски как можно раньше*
- *Обеспечить выполнение требований именно заказчиков*
- *Концентрироваться на исполняемом коде*
- *Готовиться к изменениям с самого начала*
- *Создавать систему из компонент*
- *Разработать архитектуру как можно раньше*
- *Работать единой командой*
- *Сделать качество основной идеей*

16. RUP: Фаза «Начало».

<https://youtu.be/WQjJZyV4eNw?si=8zV57XYycEwLUxA3&t=750> (22 стр)

В фазе Начало — оценить проект, понять, сколько на него нужно потратить ресурсов и времени, какие проблемы пользователей и при помощи какой функциональности он сможет решить. Формируется артефакт "Концепция" (Vision).

Цели:

- **Определение границ проекта**, что делаем, а что нет, т.е, ограничение области применения разрабатываемого ПО.
- Разработка и описание основных **сценариев использования** системы. Необходимо выяснить, как будут использовать систему пользователи, и определить для каждого сценария последовательность действий. Например, какова последовательность действий в сценарии отправки сообщения в социальной сети?
- Предложение **возможных технических решений** — на основе каких технологий, API, сторонних решений будет разработано ПО.
- Подсчёт **стоимости** и разработка **графика работ**.
- **Оценка рисков и подготовка окружения**, с помощью которого будет проводиться разработка.

Любая фаза в RUP заканчивается вехами. **Веха** — это момент времени для принятия решения о дальнейших действиях: переход на следующую фазу, либо проведение каких-либо дополнительных работ на данной фазе. Решение принимается заинтересованными сторонами (stakeholders).

Стейкхолдеры или заинтересованные стороны — лица, материально заинтересованные в создании ПО, такие, например, как инвесторы, руководство высшего звена заказчика и исполнители работ. Они хотят разработать программный продукт и получить с этого прибыль в той или иной форме. Обычные пользователи не являются заинтересованными лицами, так как для них ПО — инструмент работы, а не источник дохода.

На вехе "Lifecycle Objects" заинтересованные лица пришли к **согласию в оценке сроков, первоначальной стоимости, требованиях, приоритетах и технологиях**. После этого производится переход на следующую фазу разработки.

17. RUP: Фаза «Проектирование».

<https://youtu.be/WQjJZyV4eNw?si=cP1xoUVu-tYyNU-G&t=958> (стр 23)

На фазе Проектирование основная задача — разработка и тестирование стабильной и неизменной архитектуры системы, создание одного или нескольких прототипов системы, которые определяют исполняемую архитектуру.

Цели

- Финализировать базовую архитектуру системы
- Разработать прототип(ы) на основе архитектуры
- Убедиться в том, что архитектура, планы и сроки стабильны, риски разработаны и учтены

- Продемонстрировать, что в архитектуре можно будет реализовать требования с разумными сроками и стоимостью

Исполняемая архитектура — это полностью законченные на базе выбранных технологий несколько характерных функций разрабатываемой системы. Объём ее рекомендуемой реализации определяется на основании внесения новых архитектурных элементов каждым анализируемым требованием, и, как только реализация новых требований перестает создавать дополнительные элементы архитектуры, можно считать, что один из возможных способов построения готов.

Тестирование на этом этапе предназначено для проверки основных нефункциональных требований к системе, например, пропускной способности или времени отклика.

На основании архитектуры, прототипов и результатов тестирования уточняются планы разработки и производится переоценка и контроль рисков, уточняются сроки и стоимость системы.

Веха «Lifecycle architecture»

- Концепция, требования, архитектура проекта стабильны?
- Сформированы критерии тестирования прототипов?
- Тестирование прототипов показало отсутствие основных рисков?
- Планы разработки подробны и приемлемы по цене?
- Соотношение запланированных и затраченных расходов приемлемо, стороны подтверждают выполнимость проекта?

Веха "Lifecycle Architecture" — веха стабильности архитектуры. Контрольные точки на данной вехе исходят из того, что стоимость и сроки разработки согласно требованиям, определенным ранее, соблюдены или могут быть приемлемы с точки зрения заказчика. Контроль подлежат потраченные ресурсы и средства, что важно с точки зрения планирования следующей фазы.

Заказчик может принять и аргументированный перерасход средств, который может возникнуть после разработки архитектуры и проверки её характеристик. Сложность такой аргументации заключается в том, что стейкхолдеры со стороны заказчика — люди с бизнес-образованием, далёкие от глубокого понимания деталей современных технологий.

18. RUP: Фаза «Построение».

<https://youtu.be/WQjJZyV4eNw?si=VhUjinE2eVLKxhpH&t=1320> (стр 24)

Цели:

- Экономически эффективно, с надлежащим качеством, так быстро, как возможно, разработать продукт

- Итеративно и инкрементально провести анализ, проектирование, разработку и тестирование продукта, создать необходимые выпуски (альфа, бета...)
- Подготовить продукт, места установки и пользователей к использованию

Веха «Initial Operational Capability»:

- Достаточно ли стабилен выпуск для передачи пользователями?
- Все стороны готовы к передаче продукта пользователям?
- Соотношение запланированных и затраченных расходов все еще приемлемо?

Как было отмечено ранее, до фазы Построения должна быть разработана архитектура приложения; внесение в неё кардинальных изменений к этому моменту должно быть исключено.

Экономическая эффективность означает в том числе и то, что ресурсы команды разработки не должны тратиться на неосновные работы и переделки. Изменений и улучшений в архитектуре вноситься уже не должно, или они должны быть минимальными. Кроме того, должна проводиться постоянная работа по снижению издержек команды. Необходимо распределять работы с учетом личных особенностей и возможностей всех участников.

Во время фазы происходит создание необходимых выпусков продукта, предусмотренных планом проекта. Проводятся плановые демонстрации версий заказчику.

В конце фазы производится подготовка продукта к передаче заказчику, обследование места установки, разработка учебных материалов и обучение пользователей. Особое внимание уделяется организации и проведению тестирования согласно ранее разработанному плану тестирования, в котором описаны не только проводимые тесты на модульном, интеграционном и системном уровне, но и метрики качества ПО (например, количество вновь выявленных дефектов за неделю) и способы их получения из внутренних систем разработчиков. С точки зрения трудозатрат, на этой фазе непосредственно создание кода составляет лишь часть от полных затрат проектной команды (от 20% до 40%).

На вехе «Initial Operational Capability» принимается решение о возможности внедрения продукта на стороне заказчика. Необходимо учитывать стабильность выпуска (на основании метрик) и готовность пользователей, причём как техническую, так и с точки зрения обучения. Как и в предыдущих фазах ещё раз проверяется приемлемость соотношения реальных и запланированных затрат.

19. RUP: Фаза «Внедрение».

<https://youtu.be/WQjJZyV4eNw?si=Tz4vfojAnBcZcSAU&t=1592> (стр 25)

Цели:

- Провести бета-тестирование, сравнить работоспособность старой и новых версий
- Перенести продуктивную БД, обучить пользователей и поддерживающий персонал
- Запустить маркетинг и продажи
- Отладить процессы устранения сбоев, дефектов, проблем с производительностью
- Убедиться в самодостаточности пользователей
- Провести (совместно со всеми заинтересованными сторонами) открытый анализ соответствия разработанного продукта исходной концепции

Веха «Product Release»

- Пользователи удовлетворены?
- Финальное соотношение запланированных и затраченных расходов приемлемо?

Фаза Внедрение предназначена для запуска продукта в продуктивное использование и финального подтверждения пользователями пригодности продукта для их практических нужд.

Если система является "заказной", то есть разработанной для конкретного заказчика, то специальная группа разработчиков занимается установкой и настройкой оборудования и ПО системы. При этом может потребоваться конвертация и перенос данных из старой системы в новую, что может быть достаточно трудоёмко.

Если продукт предназначен для конечного пользователя, который сам будет устанавливать ПО у себя на компьютере, на данной фазе необходимо предусмотреть всё, что связано с будущими продажами продукта. Формирование каналов сбыта, подготовка специалистов по продажам, маркетинг и реклама, упаковка коробочных версий продукта — вот неполный список задач, с которыми придётся столкнуться разработчикам, а если это стартап, то данная фаза критически важна для начала использования продукта потребителем и, соответственно, получения прибыли.

Основной вопрос на вехе "Product Release" — удовлетворены ли пользователи? *Существует поверье, что если пользователи удовлетворены, то с окончательной оплатой продукта, и, соответственно, с получением премий проблем бывает существенно меньше.*

Кроме того, важно провести работу над ошибками, и оценить, что в процессе разработки было хорошо, а что требует исправлений. Особый контроль

подлежат затраты на разработку — необходимо оценить их отношение к спланированным в фазе Начало для формирования корректировок в будущих проектах.

20. Манифест Agile (2001).

<https://youtu.be/WQjJZyV4eNw?si=aM2S2sAG1xi-fk0n&t=3780> (стр 36)

Основной акцент в Agile-подходе ставится на требования заказчика, и, следовательно, реакция на них. Данный подход невозможен, или практически невозможен, если на проект выделяется фиксированный бюджет, без возможности его расширения.

Agile-методологии хорошо работают для внутренних проектов разработки в компаниях,

которые занимаются различным бизнесом, приносящим доход, когда заказчик непосредственно покупает время разработчиков. Разработчики, со своей стороны, должны постоянно демонстрировать результат работ, этот результат оценивается,

разработчики получают деньги, и немедленно начинают разрабатывать новый функционал.

При этом важно открыто и искренне сотрудничать с заказчиком, принимать во внимание его замечания.

Ещё одним важным принципом является максимальное сокращение расходов на труд

разработчиков, не относящийся к созданию кода. В небольших проектах с типовой

архитектурой это реализуется просто, а в более сложных порождает проблемы, так как без

моделей и документирования архитектуры невозможно создать проект, выходящий за рамки

типового.

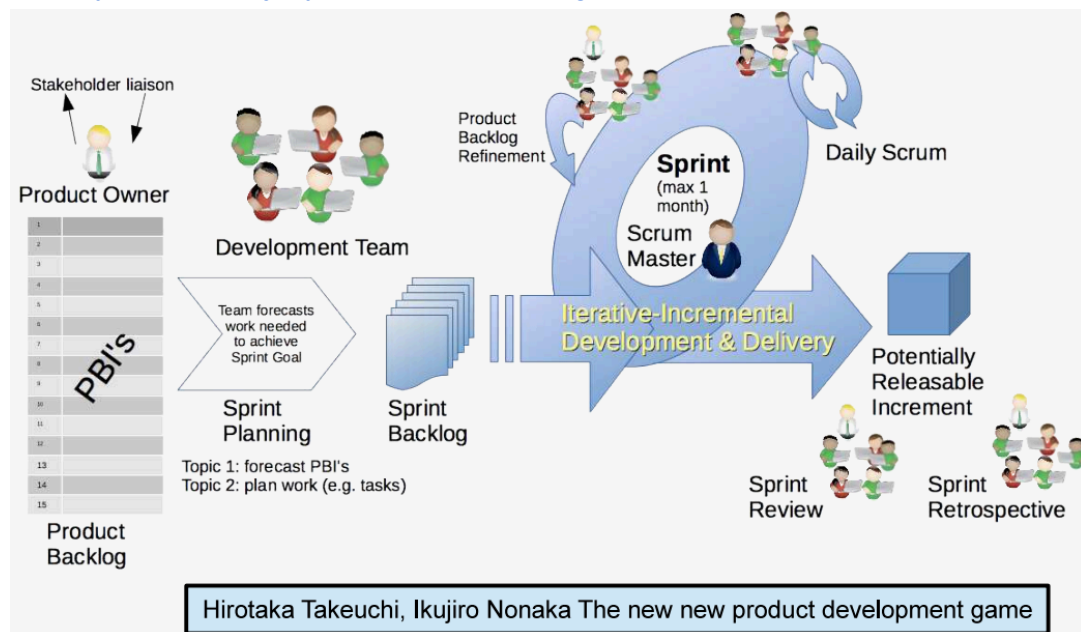
12 принципов:

- Наивысшим приоритетом для нас является удовлетворение потребностей заказчика, благодаря регулярной и ранней поставке ценного программного обеспечения.
- Изменение требований приветствуется, даже на поздних стадиях разработки. Agile-процессы позволяют использовать изменения для обеспечения заказчику конкурентного преимущества.
- **Работающий продукт следует выпускать как можно чаще и раньше**, с периодичностью от пары недель до пары месяцев.
- На протяжении всего проекта разработчики и представители бизнеса должны ежедневно работать вместе.
- Над проектом должны работать мотивированные профессионалы.
- Непосредственное общение является наиболее практичным и эффективным способом обмена информацией как с самой командой, так и внутри команды.

- **Работающий продукт — основной показатель прогресса.**
- Инвесторы, разработчики и пользователи должны иметь возможность поддерживать постоянный ритм бесконечно. Agile помогает наладить такой устойчивый процесс разработки.
- Постоянное внимание к техническому совершенству и качеству проектирования повышает гибкость проекта.
- Простота — искусство минимизации лишней работы — крайне необходима.
- Самые лучшие требования, архитектурные и технические решения рождаются у самоорганизующихся команд.
- Команда должна систематически анализировать возможные способы улучшения эффективности и соответственно корректировать стиль своей работы.

21. Scrum.

<https://youtu.be/WQjJZyV4eNw?si=7eHYz5ghKVcrpl5r&t=4066> (стр 38)



Основной и единственный служебный артефакт Scrum — **бэклог** (долг) — упорядоченный по приоритетам список требований с оценкой трудоёмкости разработки. Существуют **бэклог продукта**, обычно более общий и состоящий из бизнес-требований, и **бэклог спринта** — более детальный, с учетом технических особенностей реализации. Другим артефактом является **инкремент продукта**.

Спринт — время от 1 до 4 недель, за которое разработчики реализуют выбранный набор требований из бэклога спринта. Каждый спринт заканчивается демоверсией, и проделанная работа демонстрируется заказчику. Также через несколько спринтов проводятся ретроспективы, в рамках которых может проводиться работа над ошибками и

перераспределение обязанностей для более эффективного использования каждого участника команды.

Команда в Scrum небольшая, от 3 до 10 человек (больше плохо, т.к. всё взаимодействие должно идти устно, а в большой команде это невозможно реализовать).

Product Owner — это человек, (которые понимает целиком, что происходит) определяющий порядок разработки требований из бэклога.

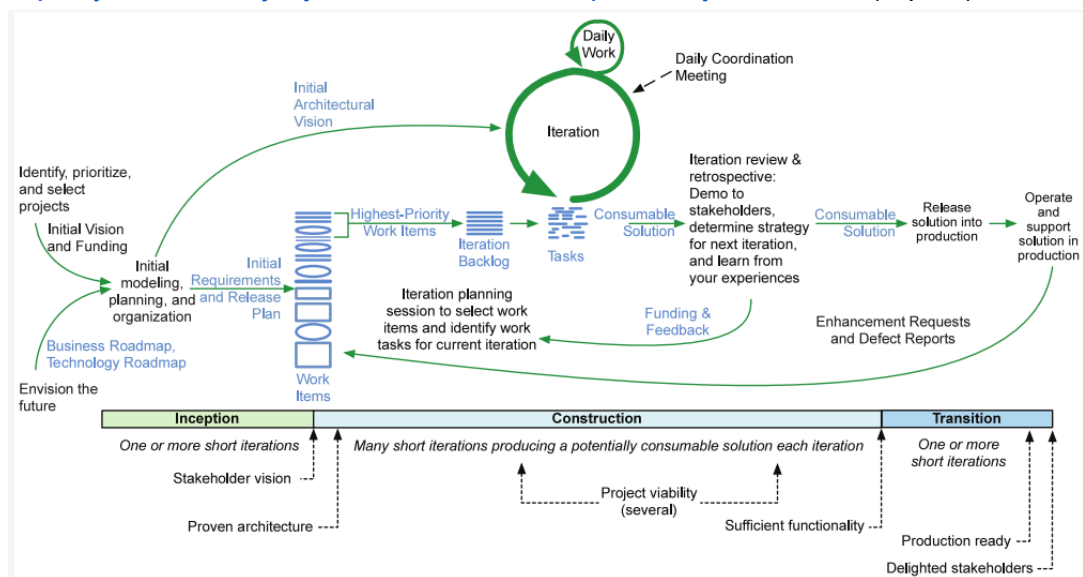
Для каждого спринта задачи конкретизируются и передаются на разработку команде. Кроме этого, отдельно выделяется **Скрам-мастер**, ответственный за проведение скрам-митинга, который помогает команде планировать спринт и следит за внутренними отношениями в команде.

Ежедневно утром производится **скрам-митинг**, где каждый отчитывается о проделанной работе, возникших проблемах, и о том, что он собирается сделать к следующей встрече.

Достоинство Scrum — простота, минимум административной работы и документов, и максимальная концентрация на работоспособном коде.

22. Disciplined Agile 2.X (2013).

<https://youtu.be/WQjJZyV4eNw?si=6Qfm3dXpVeYBNyV2&t=4667> (стр 39)



Проблема гибких методологий - невозможность масштабироваться на команды с большим числом разработчиков.

AUP (Agile Unified Process) + EUP (Enterprise Unified Process — метод разработки с учетом полного жизненного цикла ПО, включая фазы поддержки и вывода из эксплуатации) = Disciplined Agile 2.X (DAD). Подход к делению на фазы и дисциплины в DAD очень похож на RUP, но основной цикл разработки построен на базе гибких методов, в том числе Scrum.

Пока данная методология является относительно новой для рынка и находится в стадии развития.

Помимо деления процесса на фазы, которых предлагается три (Начало, Построение и Внедрение), и описания каждой роли в разработке, DAD рассматривает процессы, выходящие за рамки собственно процесса разработки. К ним относятся: управление архитектурой и повторным использованием кода, управление персоналом, служба поддержки и текущих операций компании-разработчика, управление портфолио компетенций, непрерывное улучшение процессов разработки и вспомогательных процессов, и многое другое.

23. Требования. Иерархия требований. Примеры формулировок на разных уровнях.

<https://youtu.be/Lnd-BW7IOBQ?si=hSK8ebI2WQvFByWY&t=1075> (стр 41)

Требования — это условия или возможности, которым должна соответствовать система.

Требования могут общими, или подробно описывать то, что должно быть реализовано.

Первый способ описания требований — это модели требований.

Для их описания существует шаблон — **Software Requirement Specification**, который обычно и

является формальным техническим заданием на разработку. В дополнение к нему существует

отдельный набор детализированных описаний прецедентов

(последовательность шагов по выполнению действий пользователя со всеми возможными ветвлениями в сценарии работы, включая ошибочные, прототипы графических (или иных) интерфейсов взаимодействия с системой)

Второй способ — модели прецедентов (**Use Case Model**), описывающие требования в виде

UML-диаграммы.



Иерархия по степени детализации:

- **Потребности** заинтересованных лиц. В большинстве случаев они представлены в виде пожеланий для решения конкретной задачи — "Мы хотим, чтобы нашему бизнесу было хорошо!", "Мне нужна сковородка, и чтобы на ней не пригорала еда".

Вместе с пожеланиями заинтересованные лица предоставляют набор артефактов, которые можно использовать для иллюстрации информации, которую будет содержать система.

- Задача аналитика — перейти к области решения задачи и преобразовать потребности в набор **функций**, которые должна реализовывать система. Здесь уже необходим больший формализм и чёткость в формулировках, однако эти требования всё ещё должны быть сформулированы языком, понятным для заказчика. Например: "Сковородка должна иметь антипригарное покрытие и термоизолирующую ручку".
- После утверждения заказчиком функций системы можно переходить к планированию реализации и, **требованиям к ПО**. Например: "Диаметр сковородки должен быть 25 сантиметров, величина бортиков 5 сантиметров, она должна быть выполнена из пищевого сплава алюминия с кремнием марки АК7П (7% кремния) и строго соответствовать требованиям ГОСТа 1583-93. Для антипригарного покрытия необходимо использовать покрытие фирмы Scandia толщиной 18-25 микрон."

24. Свойства и типы требований (FURPS+).

<https://youtu.be/Lnd-BW7IOBQ?si=t4OS5VVIDE50KsLf&t=1675> (стр 43)

Требование должно быть **однозначным** и **полным**.

Требования **не должны противоречить** друг другу.

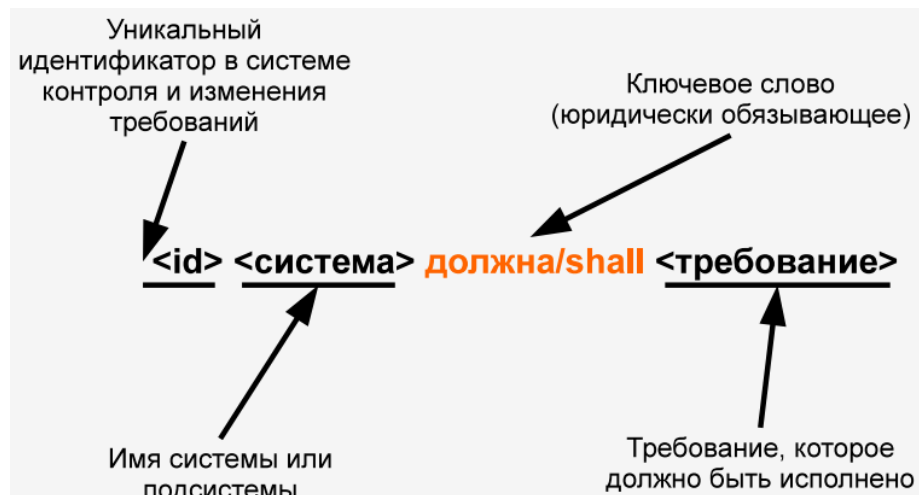
Некорректно сформулированные требования могут привести к путанице из-за того, что одним и тем же словам разные люди придают разный смысл. Для определения последовательности реализации требования должны иметь **приоритет**.

Иногда приоритет может зависеть от технической последовательности реализации — например, сковородку надо сначала отлить, а потом уже покрыть антипригарным покрытием.

При разработке всегда необходимо понимать, на основании каких потребностей или технических особенностей требование сформулировано, чтобы его можно было **проверить**.

Модификация требований также должна быть контролируемой, в любой момент необходимо знать историю их изменений.

Отслеживание требований и их модификаций производится при помощи систем контроля над изменениями, которые в современной программной индустрии достаточно развиты.



В RUP для описания и разработки требований используется модель FURPS+ (Functional, Usability, Reliability, Performance, Supportability).

Требования делятся на два типа:

- **Функциональные**, которые определяют, что система должна делать.
- **Нефункциональные**, описывающие характеристики и ограничения, накладываемые на систему.

Нефункциональные требования включают в себя:

- Требования к пользовательским характеристикам ПО
- Требования к надёжности
- Требования к производительности (например, способность обеспечить заданное время отклика или скорость обработки задач).
- Требования к условиям поддержки (например, способность проводить обновления ПО без остановки функционала).

Кроме того, требования могут включать в себя ограничения на использование тех или иных прикладных программных интерфейсов и продуктов для формирования архитектуры, требования к реализации (например,

математический метод решения задачи), а также физические требования (например, параметры импульсов).

25. Формулирование требований. Функциональные требования.

https://youtu.be/Lnd-BW7IOBQ?si=LllwCOlr_lxdaxh&t=2155

Функциональные требования определяют:

- Feature sets — наборы функциональных требований.
- Capabilities — возможности ПО.
- Security — Требования к безопасности.

FR0 «Система должна обеспечивать ввод, модификацию и удаление данных о клиенте».

SEC0 «Система должна обеспечивать двухфакторную аутентификацию пользователей с помощью имени пользователя и пароля и подтверждения с помощью СМС».

Feature Set (набор функциональных требований) — это набор свойств продукта, необходимый для выполнения конкретной деятельности.

Требования к безопасности обычно описываются отдельно и включают в себя:

1. метод аутентификации,
2. список ролей, существующих в системе,
3. шифрование,
4. хранение данных в защищенных источниках.

Кроме того, данные требования могут описывать и организационные меры, исключающие доступ злоумышленников к системе.

26. Требования к удобству использования и надежности.

<https://youtu.be/Lnd-BW7IOBQ?si=a8FEnAyPEOMPFV83&t=2300>

Usability

- Human factors — учёт особенностей пользователя.
- Aesthetics — эстетические требования.
- Consistency in the user interface — согласованность пользовательского интерфейса.
- Online and context-sensitive help — требования к справочной подсистеме.
- Wizards and agents — мастера и ПО, повышающие продуктивность и простоту работы пользователя.
- User documentation — требования к пользовательской документации.
- Training materials — требования к учебным материалам.

Человеческий фактор, то есть учёт физических особенностей человека.

Например, обычные пользователи (не связанные с компьютерными в обычной жизни) в диалоговых системах ожидают от ПО поведения человека, т.е. интервал ответа системы должен находиться в диапазоне 1-5 секунд, и, если ответ приходит быстрее, человек может не понять, что-то произошло в системе, а если медленнее — то он будет считать, что система "тупит". Кроме того, для пользователей с ограниченными возможностями обычно реализуют специальные элементы интерфейса.

Эстетические требования могут отсутствовать, но, если они определены, они могут быть сложными. В крупных компаниях в обязательном порядке существуют так называемые **brandbooks** — справочники по текущему корпоративному стилю.

Отдельным пунктом при разработке могут требоваться мастера настройки, которые упрощают последовательность действий пользователя для типовых задач.

В юзабилити-требования включаются требования к онлайн-подсказкам, пользовательской документации и учебным материалам, как к средству подготовки пользователей и решения возникших у них проблем.

Для реализации таких требований на рынке существует подход, который получил название User Experience Design или UX, который предназначен для создания удобных пользовательских интерфейсов.

Reliability

- Frequency and severity of failure — частота и обработка отказов.
- Recoverability — способность системы восстанавливать продуктивное функционирование.
- Predictability — предсказуемость поведения.
- Accuracy — точность.
- MTBF — среднее время между отказами.

Требования к надёжности (**reliability**) — предназначены для фиксирования способности ПО безотказно функционировать в течение определённого периода времени.

Отказ системы — это неспособность системы выполнять основную функцию.

Сбои — случаи неспособности выполнять отдельный функционал при сохранении общей работоспособности системы. Нужно разносить типы сбоев и расставлять у них приоритеты.

В требованиях указывается допустимое число отказов и сбоев за определённый промежуток времени, а также штрафы разработчиков за превышение.

recoverability — способность системы восстанавливать продуктивное функционирование в течение заданного времени.

accuracy — точность, например, проведения вычислений. Для математических моделей управления физическими системами точность имеет решающее значение.

MTBF (Mean Time Between Failures) = Общее время работы системы /
Количество отказов за этот период

27. Требования к производительности и поддерживаемости.

<https://youtu.be/Lnd-BW7IOBQ?si=uL2wxQK2t6TTKKXE&t=3000>

Performance:

- Speed — скорость решения задач.
- Efficiency — эффективность.
- Availability — готовность системы к решению задач.
- Throughput — пропускная способность.
- Response Time — время отклика.
- Recovery Time — время восстановления.
- Resource Usage — использование системных (и других) ресурсов

Особое значение **скорость** принимает в системах реального времени, когда необходимо произвести расчеты в точно определенный интервал времени, иначе может быть нарушено функционирование всего объекта управления.

Требования к **эффективности** фиксируют процент времени, которое тратится на выполнение полезных задач, по отношению к времени на выполнение общесистемных.

availability - способность быстро начать выполнение задачи. Иногда это ещё называют реактивностью системы.

throughput показывает, какой объём данных или запросов система может обработать за единицу времени.

Supportability:

- Extensibility — расширяемость.
- Adaptability — адаптируемость под конкретные задачи.
- Maintainability — поддерживаемость.
- Compatibility — совместимость.
- Configurability — способность задавать конкретную конфигурацию.
- Serviceability — возможность проведения профилактики и обслуживания.
- Installability — требования к установке на разные системы.
- Localizability (Internationalization) — локализуемость для разных языков и географических регионов.

28. Атрибуты требований.

<https://youtu.be/Lnd-BW7IOBQ?si=vLpqcxmW3wJUzKIB&t=3525> стр 51

Требования могут быть помечены различными атрибутами, которые помогают менеджерам сортировать их

MoSCoW:

MUST have (Minimum Usable Subset) — фундаментальные для системы требования, без которых финальная реализация системы не имеет смысла. Например, современный телефон немислим без возможности позвонить другому абоненту и принимать звонки.

Should have — важные требования, которые следует реализовать при наличии времени в процессе разработки. Важным требованием, например, может быть возможность совершать звонки на наличие на одну кнопку телефона для вызова экстренных служб.

Could have — потенциально возможные: улучшающие, к примеру, пользовательское отношение к разрабатываемой системе, например, логичная структура меню телефона.

Won't have (Would like to have) — могут быть реализованы в следующих версиях системы, так как текущие временные рамки не позволяют провести их разработку.

Иным способом приоритизации требований является:

- задание им цифрового показателя приоритета — например, от 1 до 10.
- Статус: Предложенные, одобренные, отклоненные, включенные.
- Трудоемкость: Человеко-часы, функциональные точки, use-case points, «попугаи».
- Риск.
- Стабильность: Высокая, средняя, низкая.
- Целевая версия.

29. Описание прецедента.

<https://youtu.be/Lnd-BW7IOBQ?si=ne3ZEhuKGsraqFCU&t=4120>

Прецедент: PieSelling
ID: 2
Краткое описание: Бабушка продаёт пирожки.
Главные актёры: Бабушка-продавец, Клиент (или любой другой пользователь).
Второстепенные актёры: нет.
Предусловия: Клиент знает, какой пирожок он предпочитает. Бабушка проверила весь ассортимент. У клиента достаточно средств.
Основной поток: 1. Клиент обращается к Бабушке за пирожками. 1.1. Клиент называет количество и номенклатуру пирожков. 1.2. ALT1 PieRecommendation. 1.3. Бабушка подтверждает номенклатуру и называет общую стоимость.

В процессе дальнейшей разработки требований диаграммы прецедентов используются только как изначальная модель. Дальнейшее развитие прецедентов связано с тем, что требования к системе формулируют в виде текстового описания, графических интерфейсов пользователя и других способов. В качестве примера приведен прецедент PieSelling — продажа пирожков.

Ключевое слово **extend** выражается в виде альтернативного сценария.

Существует основной поток событий, где продаются именно те пирожки, и в том количестве, как это было указано клиентом. Если срабатывает условие активизации альтернативного прецедента, то в определенный момент времени вызывается сценарий, который обычно описывает дополнительные действия системы по обработке действий пользователя.

30. Риски. Типы Рисков.

Риск - сочетание вероятности события и его (негативных) последствий.(по ГОСТ ИСО/МЭК 16085-2007).

По сути “риск” - это действие, которое ведёт к потере денег, времени разработки, потери лояльности заказчика.

Типы рисков:

- Прямые:
Те на которые мы можем повлиять(Недостаточная квалификация разработчиков -> уволи... отправить на курсы повышения)
- Непрямые:

Те на которые мы не можем влиять (В сервер твоей компании прилетел украинский дрон)

- Ресурсные:

Когда нас ограничивает что-то физически(Не хватает денег, времени, люди)

- Бизнес-риски:

Всё как в обычном бизнесе - связано с конкурентами/подрядчиками, плохие решения

- Технические риски:

Возможность реализации опции, которое мы проектируем. Размывание границ проекта.

- Политические риски:

Тут имеется ввиду политика внутри компании(Пришёл новый менеджер и пытается отдать реализацию твоего проекта, на других его корешей, а потом увольняет тебя и садит сына своей бабушки по кошачьей линии)

- Форс-мажор:

Всё что связано с государством завтра + не можем влиять на это, так что на выборы даже нет смысла ходить.(20 - коронавирус, 22 - сво, 25 - мах)

С рисками мы можем делать только две вещи:

- Оценивать:

- Идентифицируем риск (Вообще понять что он существует)

- Анализируем риск (Понять откуда он и к чему это приведет)

- Приоритизация риска (Понять нужно ли его избегать или можно подзабыть)

- Контролировать и управлять:

Полностью описанный риск после оценки переходит на стадию контроля и управления.

- Планирование реакции (То как мы будем реагировать)

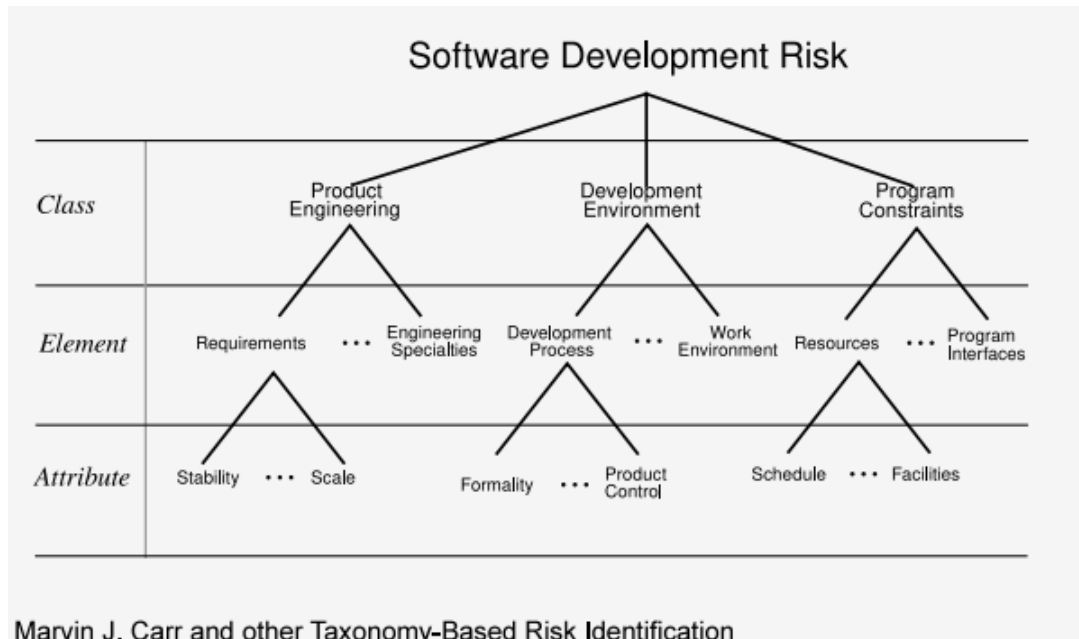
- Мониторинг

- Разрешение неопределенностей, связанных с рисками (решаем какие-то проблемы)

В RUP есть отдельный шаблон для рисков, называемый "risks list" и там более

подробно описаны все эти стадии.

31. Управления рисками. Деятельности, связанные с оценкой.



1. Идентификация рисков:

Марвин Дж. Карр в 1993 году предложил схему идентификации рисков, построенную на иерархической структуре источников риска в компании-разработчике.

1 уровень:

- Риски связанные с самой разработкой (Product Engineering)
- Риски связанные с окружением, где осущ. разработка (Development Env.)
- Риски связанные с программным обеспечением (Program Constraints)

Далее эти классы разделяются на элементы, а они в свою очередь на атрибуты, которые указывают на возможные источники возникновения рисков.

(Углубляться в каждый атрибут и элемент, на схеме, наверно нет смысла, потому что дальше всё индивидуально и зависит от самой компании, но паттерны прослеживаться будут)

Также Карр разделил риски по возможности отследить источник:

- Известные - те, которые легко идентифицировать и определить место возникновения в компании (Часть разработчиков некомпетентна в части какой-либо технологии)
- Неизвестные - те, место которых можно только предположить с помощью описанной иерархии рисков, а ранее команда разработки с таким не сталкивалась.
- Непознаваемые - те, место возникновения которых, невозможно предугадать, а способы борьбы разработать заранее нельзя (Они всегда застанут вас врасплох)

2. Анализ риска:

Есть множество методов и моделей для анализа. Примеры, которые есть на

слайдике:

Стоимостная - влияние риска на разработку, но в денежном эквиваленте;

Сетевая - строит систему связей рисков. Можно понять, как одно событие потянет

за собой целый каскад.

Качественных факторов - описывает риски не с численной стороны, а их природу,

факторы влияния и помогает расставить приоритеты.

У рисков есть параметры:

- Вероятность (процент или возможность наступления)

- Масштаб (магнитуда/сила удара)

оба параметра можно задать, как математической вероятностью, так и объемом денежных потерь. (Обычно пятибалльной шкалы бывает достаточно). Они могут быть заданы нечетко, т.к. делается это человеком и его субъективным отношением (Если ему будет по барабану, то возможно оценка занижена. Если человек горит делом, то оценка будет скорее завышена).

Риски также могут быть заданы шкалами. В таком случае формируются группы параметров. В явном виде необходимо указать приближение к наступлению риска. (Можно зафиксировать, что если заказчик не платит допустим 5 дней, то это не сильно скажется на работников. Если 6 и более, то это может вызвать негатив. Если денег не видно уже пол месяца, то разработчики точно рискуют не получить зп вовремя. Или разработчики устроят БУНТ! и не выйдут на работу)

3. Приоритизация рисков:

При приоритизации рисков, вводят понятие “Экспозиция риска”. (Risk Exposure)

Это понятие показывает потенциальные потери в виде функции вероятности появления опасного события и величины последствий его возникновения.

$$RE = Prob(UO) * Loss(UO)$$

По сути это произведение вероятности на масштаб денег, которые потеряем. Потом составляется документ “ТОР -10” со списком самых главных рисков, который следует обновлять по мере разработки. (Это круто, потому что у тебя каждый риск это фиксированное число и ты в динамике можешь отслеживать конкретные риски и добавлять новые или убирать старые)

32. Управления рисками. Деятельности, связанные контролем и управлением.

1. Планирование реакции:

Самое первое, что можно делать с риском это подумать, как мы будем на него

реагировать:

- Избегать риск (Реорганизовать проект, чтобы избежать наступление риска)
- Перенести риск (Если знаем, что риск неизбежен, то перенести его на кого-то. Если надо доп финансирование, то переложим ка это на заказчика. Нет денег - нет работы. И теперь уже по его вине проект не выйдет вовремя)
- Сокращение вероятности риска (Если знаем о недостатках технологии, то попробуем обосновать использование другой)
- Приём риска (Соглашаемся, что риск может быть. Если это новая технология, то обучаем сотрудников)

2. Разрешение риска:

Для того, чтобы риск не наступил, возможно выполнение следующих действий:

- Построение как можно большего числа прототипов системы. (Каждый новый показ прототипа налаживает связь с заказчиком и мы избегаем ситуаций “всё не нравится, переделываем”.)
- Аналитическая работа. (По сути просто работа над ошибками)
- Подбор персонала. (Индивидуальная штука для каждой компании)

3. Мониторинг рисков:

- Есть системы для непрерывного мониторинга списка, что ей дали. (Как пример, это могут быть системы контроля версий или плагины для Jira, которые отслеживают твою работоспособность, закрытие задач и т.д.)
- Руководитель составляет топ-10 и следит за рисками. Как только, что-то стало “желтым”, то бежит разбираться.

33. Изменение. Общая модель управления изменениями.

https://youtu.be/-i5zCy0_EnA?si=wBT1gvFQiwPcQ3Sp&t=40 (стр 67)

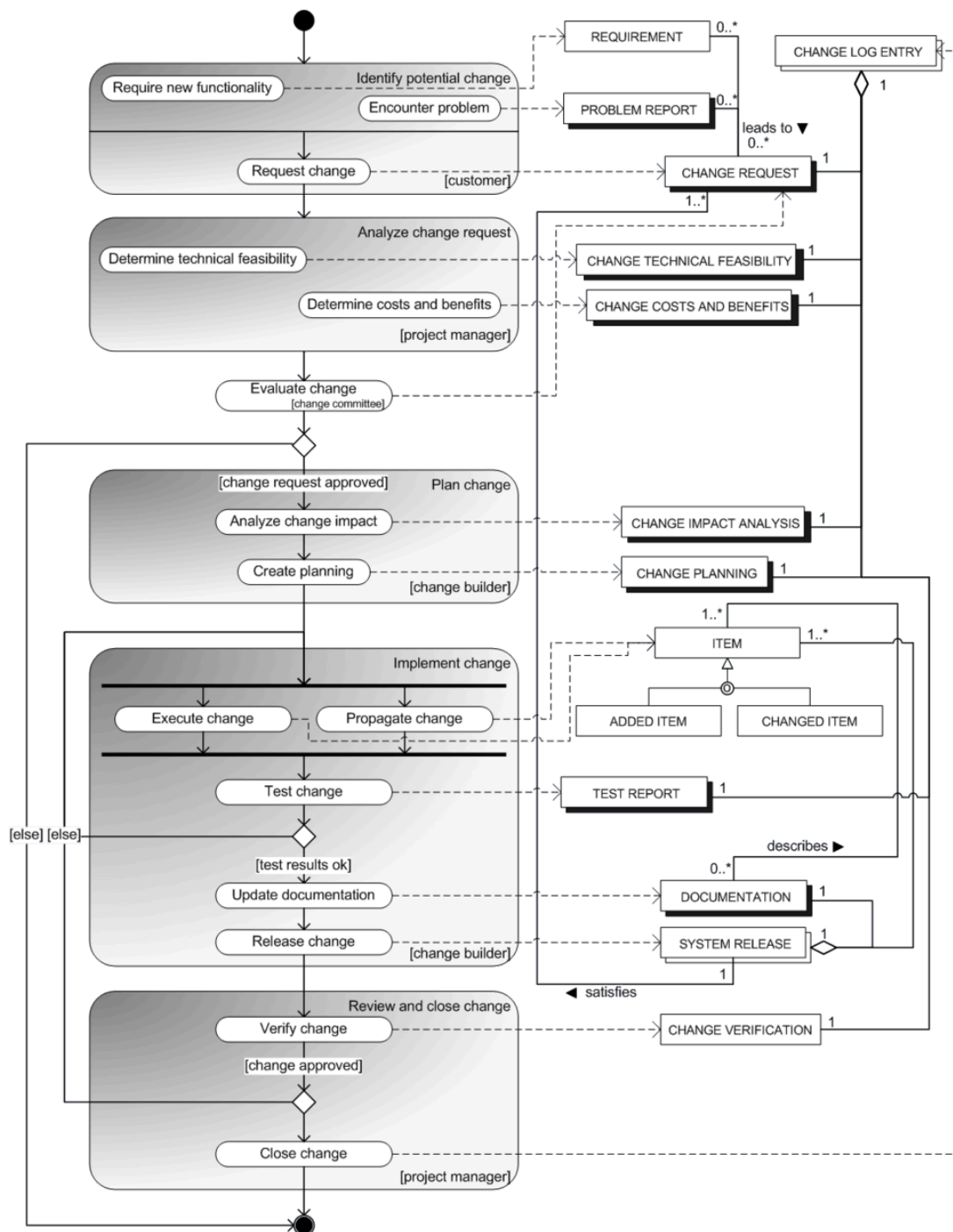
Контролируемое, журналируемое обновление системы.

Атрибуты изменения:

- Идентификатор, дата, ответственный, описание, журнал изменения.

Управлению изменениями подлежат:

- Требование новой функциональности.
- Нефункциональные требования.
- Исправление дефектов.
- Другие артефакты архитектуры, анализа, дизайна.



В левой части указаны активности (деятельности) ролей процесса с целью формирования и изменения набора артефактов для ведения изменения. Справа показаны сами артефакты изменения, участвующие или возникающие в процессе.

Заказчик (Customer) вносит изменения в систему ради появления новой функциональности (Require new functionality) или исправления выявленных им проблем (Encounter problem).

Каждая из этих деятельности приводит к созданию документа или материальных артефактов:

Requirements (требования к системе) Problem report (отчёт о проблеме или ошибке). Оба типа запросов формируют т.н. запрос на изменение (Change Request, часто используется акроним CR), и на каждый такой запрос формируются записи в Change Log Entry.

После этого запросы на изменения поступают к менеджеру проекта (Project manager). У менеджера есть две необходимых деятельности, которые он должен осуществить в процессе анализа запроса на изменения: он определяет техническую необходимость и осуществимость изменений (technical feasibility), а также анализирует стоимость и преимущества для пользователя (costs and benefits). При этом первый фактор формирует документ или артефакт Change Technical Feasibility, а второй - Change Costs and Benefits. Всё это отражается в журнале изменений.

После этого запрос поступает к т.н. комитету по изменениям, который, оценив необходимость изменений и стоимость решения проблемы, меняет статус изменения в Change Request. Если запрос подтверждён, то производится анализ и реализация изменений, в противном случае запрос может быть отменён или отложен.

После принятия решения об изменении анализируется его возможное влияние на другие части системы и возможные последствия для пользователей (Analyze change impact). Необходимо помнить, что в сложных системах внесение изменений в одну часть системы почти всегда приводит к влиянию, и соответственно, изменениям в других частях, поэтому вопрос о влиянии изменения следует тщательно изучить.

После анализа организуется планирование проведения изменений в системе (Create Planning), в ходе которого определяются ресурсы, время и график осуществления действий, необходимых для внесения изменения. В результате этих двух действий появляются артефакты Change Impact Analysis и Change Planning.

Затем изменения передаются на реализацию. Роль, ответственная за неё (Change builder), создает объекты, совокупность которых составляет реализацию изменения (Items), например, исходный код. После этого производится тестирование, вносятся изменения в связанную документацию, и выпускается новый выпуск или версия продукта, «заплатка» и/или конфигурация. При этом создаются артефакты — отчеты о тестировании (Test

Report), собственно, документация (Documentation) и выпуск системы (System Release), соответственно.

В конце процесса изменения передаются менеджеру проекта (и иногда - заказчику), проверяются (создаётся артефакт Change Verification), и формально утверждаются.

34. Системы контроля версий. Одновременная модификация файлов.

ОПИ (МиСПО) #4. Управление изменениям, системы контроля версий (стр 71)

Существуют три основных типа систем контроля версий:

На основе файловой системы. Данный подход является устаревшим. Такая система контроля создавала файлы слежения за текущей директорией и позволяла хранить и отслеживать версии только в рамках одной файловой системы. Также было возможность экспортировать файловую систему для доступа удалённых клиентов при помощи сетевых файловых систем (например, NFS).

Централизованная, с единым репозиторием — хранилищем исходным кодов проекта на сервере, и удалённым доступом клиентов по специальным протоколам. К таким системам относится Subversion.

Распределённая. В ней существует центральный репозиторий, из которого пользователи скачивают данные на свои локальные репозитории. Обратно в центральный репозиторий данные попадают после нескольких стадий локальных проверок. К распределённым системам относится Git.

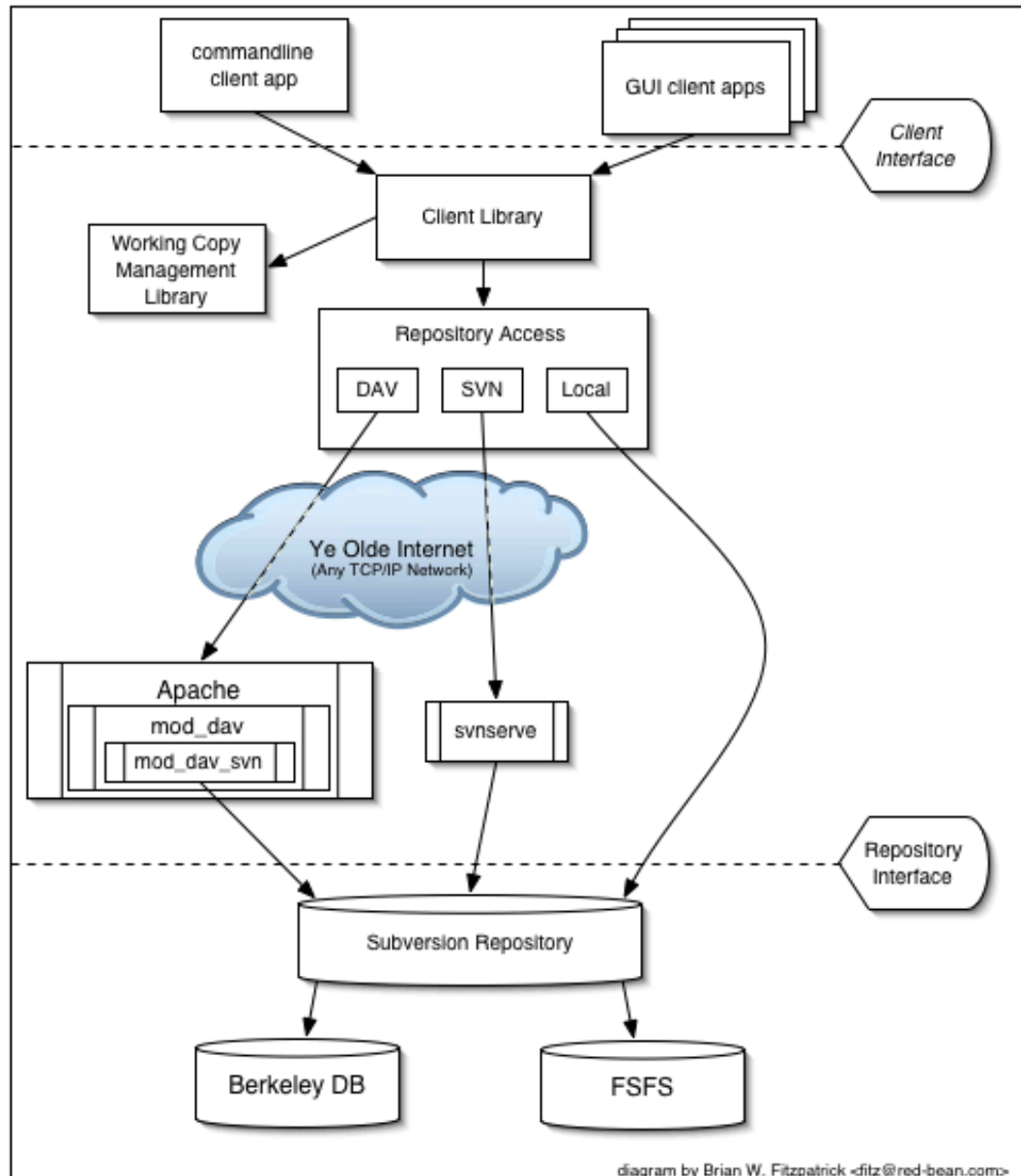
Главной проблемой при работе с СКВ является необходимость одновременного редактирования одних и тех же файлов несколькими разработчиками.

Lock-modify-unlock, где при работе одного пользователя с файлом, он блокируется для других, и они не могут его модифицировать. Это замедляет работу команды, так как другим пользователям приходится либо ничего не делать, либо переключаться на другие задачи, не связанные с заблокированным файлом.

Copy-modify-merge, где каждый пользователь копирует себе весь репозиторий, работает с ним, и изменения всех пользователей сливаются (merge). В данном случае именно слияние рабочих копий разработчиков представляет собой основную проблему. (мб привести пример)

35. Subversion. Архитектура системы и репозиторий.

ОПИ (МиСПО) #4. Управление изменениям, системы контроля версий (стр 73)



- Централизованная, репозиторий хранится на сервере.
- Доступ к репозиторию:
 - Svnserve.
 - Apache + модуль mod_dav_svn.

уровень хранения репозитория (Subversion repository), который может иметь две технические реализации — размещаться в базе данных Berkeley DB, или, в простейшем случае, в файловой системе (FSFS).

Доступом к репозиторию управляет демон **svnserve**, который получает команды пользователя и выполняет соответствующие изменения в репозитории. Вторым вариантом доступа является использование **сервера Apache** и его модулей, реализующих ту же логику, что и svnserve.

Удалённый доступ к репозиторию может осуществляться по нескольким протоколам. Обычно используется протокол svn или связки svn+https и ssh+svn.

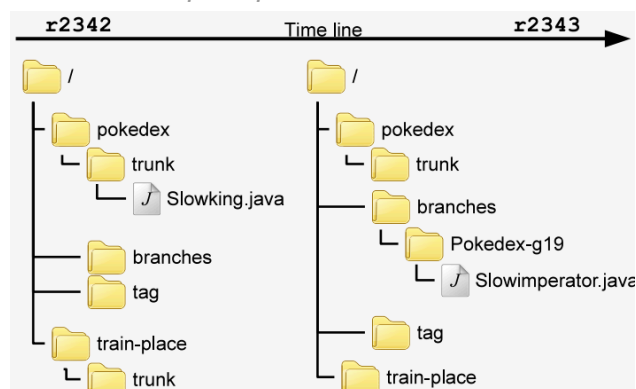
Репозиторий — это набор файлов проекта(ов), организуемый определённым иерархическим образом для удобной работы с проектом.

В SVN каждая фиксация изменений (svn commit), пришедшая в репозиторий, повышает версию этого репозитория на 1.

Ревизия репозитория — это всегда целое число, уникально характеризующее набор файлов, лежащих в репозитории на какой-то момент времени, задаваемый номером ревизии. В каждый момент времени, при условии успешной фиксации изменений, репозиторий является целостным.

Каждая фиксация изменений svn обязана включать файлы, изменения которых должны быть помещены в репозиторий. Если такое изменение производится из командной строки, то эта строка должна содержать все файлы.

В качестве примера:



Организация файлов обычно такова: существуют каталоги проектов (здесь pokedex и train-place), внутри которых размещены директории, предназначенные для организации работы с разными версиями проекта.

В каталоге trunk происходит основной процесс разработки. В этот каталог разработчики ежедневно вносят результаты своего труда, оформленные в виде целостного набора изменений (с точки зрения и набора файлов, и их логического содержания). В случае необходимости поставки различных версий разным заказчикам, либо с целью фиксации истории релизов, стабильные на некоторый момент времени копии могут формироваться на основе содержимого каталога trunk и копироваться в дополнительные каталоги branch и tags.

1. trunk — основная ветвь разработки.

2. branches — хранение модификаций продукта:
 - a. Releases — значимые версии продукта (3.0, 3.5).
 - b. Features — выполнение работ над версиями со существенными изменениями без влияния на trunk
 - c. Vendor — версии с модификациями сторонних библиотек. Vendor drop.
3. tag — функционально целостные изменения:
 - a. Исправления дефектов ПО (3.5.0-B2947219).
 - b. Минорные версии (с исправлением нескольких дефектов — 3.5.0.1).

36. Subversion: Основной цикл разработчика. Команды.

📺 ОПИ (МиСПО) #4. Управление изменениям, системы контроля версий (стр 75)

Основной цикл разработчика

svn checkout — первоначальное создание рабочей копии.

Ночь! Вечер День Утро
↓

- Обновить рабочую копию:
 - **svn update**
- Изменить необходимые файлы:
 - **svn (add, delete, copy, move, mkdir)**
 - Просмотр изменений: **svn status** и **svn diff**
 - Откат изменений: **svn revert**
- Фиксация изменений на сервере:
 - **svn update** для загрузки изменений других.
 - Разрешить конфликты содержимого и структуры файлов.
 - **svn commit** — собственно фиксация.

Одна из задач менеджера распределить задачи, чтобы минимизировать кол-во конфликтов.

При этом, тому, кто делает первый коммит, не нужно разрешать каких-либо конфликтов, так как он загружает на сервер первое изменение. Также следует отметить, что конфликты между версиями разных разработчиков могут привести и к конфликтам внутри команды. Существует также

понятие ночного билла (Nightly build), который собирается именно на основе вечерних коммитов.

37. Subversion: Конфликты. Слияние изменений.

У нас есть конфликты **содержимого** и структуры:



Конфликты содержимого файлов

- (e) edit — изменить файл в редакторе.
- (df) diff-full — показать список несовпадений между версиями.
- (r) resolved — подтвердить, что конфликт разрешен в текущей (м.б. отредактированной) версии файла.
- (dc) display-conflict — показать все конфликты.
- (mc) mine-conflict, (tc) theirs-conflict — подтвердить мои (или из репозитория) версии для всех конфликтов.
- (mf) mine-full, (tf) theirs-full — подтвердить версии полностью для файла.
- (p) postpone — отложить «на потом».
- (l) launch — запустить внешнее средство.

```
$ svn update
Updating '.':
U    Pokedex.xml
G    Slowimperator.java
Conflict discovered in 'Slowking.java'
Select:
```

filename.mine — до update
filename.rOLDREV — до правок
filename.rNEWREV — в репозитории

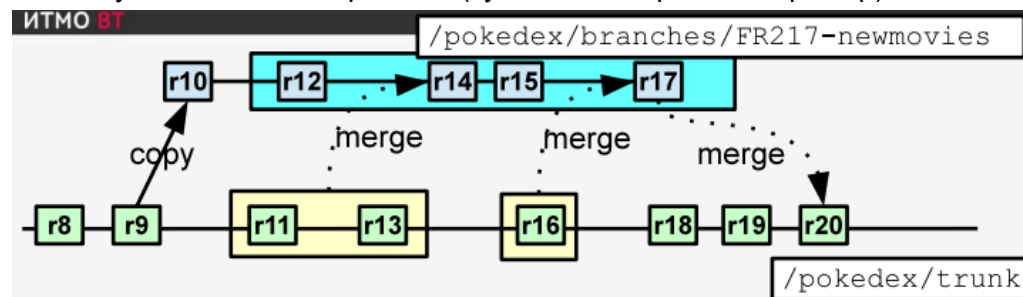
Чтобы их решить можно воспользоваться diff посмотреть в чём отличие, согласовать с автором прошлой версии.

Структуры:

Конфликты такого рода происходят тогда, когда, например, в репозитории перемещаются файлы, а в локальной рабочей копии в эти же файлы вносятся изменения. Иными словами, не совпадает структура программ, и файлы, которые были просто отредактированы на локальной копии, в репозитории переименованы, перемещены, или даже удалены.

Для **слияния** изменений мы используем `svn merge`, в ходе которого могут

возникнуть такие же конфликты. (тут неплохо привести пример)



- `svn merge` — подгружает изменения из веток в рабочую копию, учитывая изменение структуры:
 - `^/branches/FR217-newmovies` — ветвь источника.
 - `-r10:HEAD` — ревизии репозитория для интеграции.

38. GIT: Архитектура и команды.

ОПИ (МиСПО) #4. Управление изменениям, системы контроля версий (стр 81)



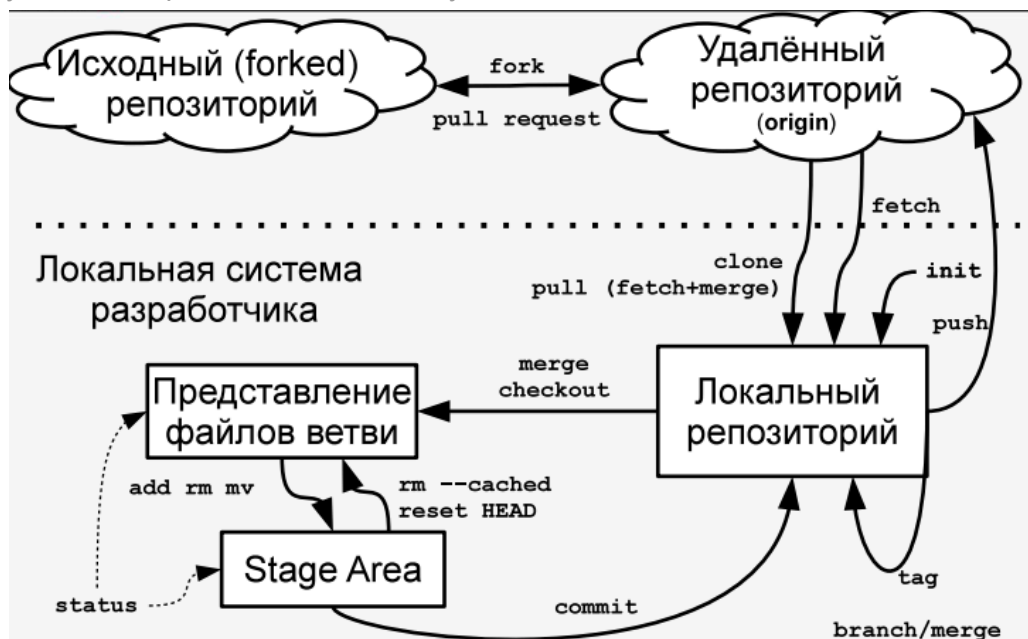
Изначальная задумка в том, что нет единого общего хранилища и каждый у себя хранит все необходимые файлы. Каждый локальный репозиторий имеет свой URL, по которому он может быть доступен, а другие разработчики могут обращаться к нему при помощи алиасов (синонимов), что упрощает команды передачи изменений от разработчика к разработчику и от репозитория к репозиторию.

Благодаря такой архитектуре рабочий процесс совместной работы может быть адаптирован для каждой конкретной команды. три основных рабочих процесса:

централизованный рабочий процесс — где существует единственный централизованный репозиторий, где все разработчики синхронизируют свою работу с ним;

рабочий процесс с менеджером по интеграции — где каждый разработчик изменяет канонический репозиторий в своей локальной копии, которую он открывает для других для чтения, а после завершения правок запрашивает интеграционного менеджера загрузить правки из своей копии для чтения в канонический репозиторий;

рабочий процесс с диктатором и лейтенантами - в основном такой подход используется на огромных проектах, насчитывающих сотни участников; самый известный пример - ядро Linux. Лейтенанты - это интеграционные менеджеры, которые отвечают за отдельные части репозитория. У всех лейтенантов есть свой интеграционный менеджер, который называется доброжелательным диктатором. Репозиторий доброжелательного диктатора выступает как эталонный, откуда все участники процесса должны получать изменения.



GitHub, предоставляет рабочий процесс с интеграционным менеджером. В нем существует центральный удалённый репозиторий, в котором сохраняются результаты работы всей команды разработчиков. Центральный репозиторий может быть получен при помощи организации ветви (fork) от любого другого известного проекта.

Такому центральному репозиторию по умолчанию git назначает алиас origin. Центральный репозиторий для каждого из разработчиков команды является удалённым.

У каждого разработчика есть свой локальный репозиторий, который является клоном (копией) удалённого репозитория. Разработчик выбирает необходимую ветвь в локальном репозитории, получая представление на файловой системе файлов с содержимым ветви, которые он может изменять.

Во время работы с рабочей копией разработчику указывает, какие файлы включить в фиксацию изменений. Эти изменения помещаются в т.н. **Stage Area** — специальную область, из которой файлы будут включены в коммит. В отличие от `svn`, здесь нет необходимости указывать все файлы, участвующие в фиксации изменения в одной команде — в `git` их можно добавить за несколько операций. Затем изменения фиксируются в локальном репозитории. Для помещения информации в центральный репозиторий разработчик использует команду `push`, при которой, естественно, могут возникать конфликты.

Другой способ фиксации изменений в удалённом репозитории — это конфигурация запросов на фиксацию изменений (`pull requests`). При этом изменения, которые разработчику необходимо записать, сначала должны быть подтверждены владельцем ветви. Таким образом обычно происходит слияние функциональности в открытом программном коде, управляемым сообществом разработчиков.

Локальный репозиторий

- **init** — создать новый репозиторий в текущей папке (появляется `.git`).
- **clone** — клонировать существующий удалённый репозиторий к себе локально.
- **fetch** — забрать новые коммиты с удалённого репозитория, но не вливая их в текущую ветку.
- **pull** (`fetch+merge`) — забрать изменения с удалённого и сразу слить их в текущую ветку (`fetch + merge`).
- **push** — отправить локальные коммиты в удалённый репозиторий (`origin`).
- **tag** — создать (или удалить) метку версии на конкретном коммите.
- **branch/merge** — `branch` управляет ветками (создание, удаление, список), `merge` сливает указанную ветку в текущую.

Stage Area / индексация

- **add** — добавить изменения файла(ов) в индекс (`staging`), подготовить к коммиту.
- **rm** — удалить файл из репозитория; с `--cached` — перестать отслеживать, но оставить файл на диске.
- **mv** — переименовать или переместить файл, фиксируя этот факт в истории `Git`.
- **reset HEAD** — убрать файлы из индекса, вернув их в «просто изменённые» (`unstaged`).
- **rm --cached** — специальный вариант `rm`, который убирает файл из отслеживаемых, не трогая рабочую копию.

Представление файлов ветви / рабочее дерево

- **status** – показать текущее состояние: какие файлы изменены, какие добавлены в индекс, какие не отслеживаются.
- **checkout** – переключиться на ветку или конкретный коммит, обновив рабочие файлы под выбранное состояние.
- **merge** – слить указанный бранч/коммит в текущую ветку, создав merge-коммит (если нужны).

Коммиты

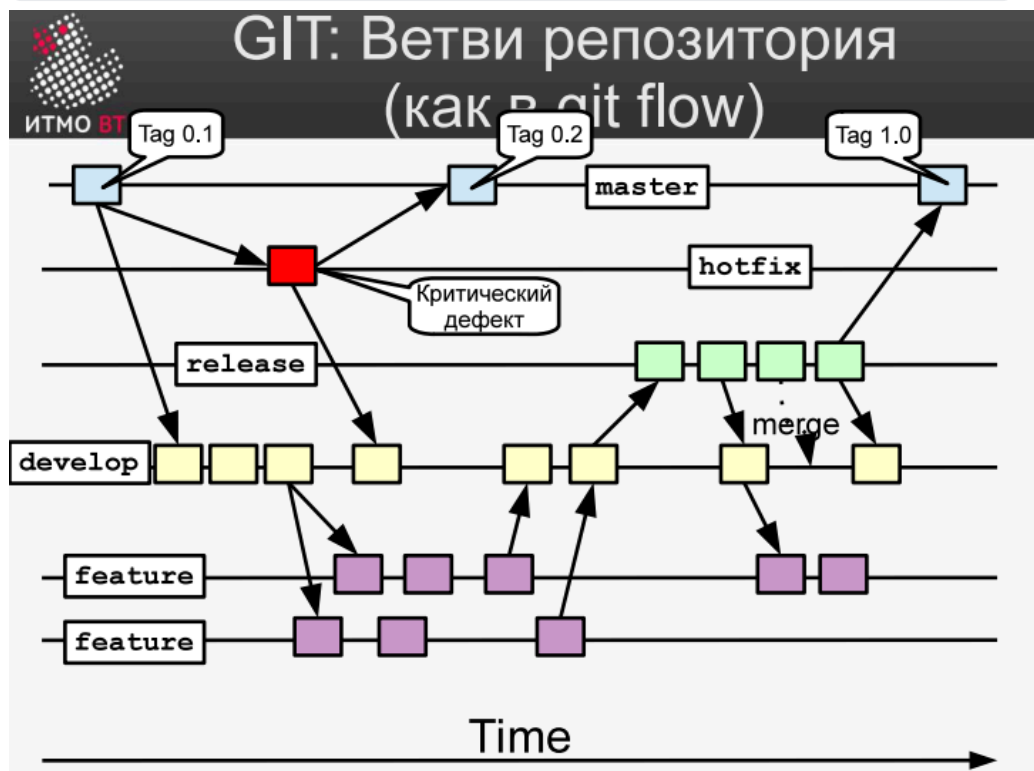
- **commit** – зафиксировать содержимое индекса в истории как новый коммит с сообщением.

Удалённый репозиторий и форки

- **fork** – операция на GitHub/подобных сервисах: создать свою копию чужого репозитория (исходный → ваш).
- **pull request** – запрос на вливание ваших изменений из форка/ветки обратно в исходный репозиторий, через веб-интерфейс.
- **origin** – стандартное имя основного удалённого репозитория, с которым связан локальный (`git remote add origin ...`).

39. GIT: Организация ветвей репозитория.

ОПИ (МиСПО) #4. Управление изменениям, системы контроля версий



В Git создавать ветку принято на почти любое самостоятельное изменение кода (и это считается best-practice). Ветки в гите легковесные — это просто указатель из 40 символов, являющиеся ничем иным как хеш коммита в древовидной структуре репозитория.

В ветви Master находятся версии, готовые к поставке пользователю. Эти версии являются основными версиями продукта. Функционал там чётко определен, и список дефектов известен.

Основная разработка происходит в ветви Develop. Данные ветви используют Master как базовую, и в них происходит разработка новой версии продукта. После кодирования все разработчики сливают свои изменения именно в ветку Develop. При этом изменения рекомендуется делать по функциональным требованиям продукта. Для каждого из требований создается своя ветвь Feature, и там аккумулируются коммиты для этого требования. По мере завершения разработок функционала коммиты из Feature-ветвей подгружаются в ветку Develop.

Когда накоплено достаточное количество функционала для выпуска новой, готовой к поставке версии, изменения из Develop формируют ветвь Release, а сама ветвь Develop остаётся для дальнейшей разработки новых функций. Ветвь Release используется для исправления тех дефектов, которые препятствуют выходу новой версии. После исправлений готовая версия продукта копируется в ветку Master. При этом исправления дефектов сливаются в ветвь Develop, так как они там также будут необходимы в дальнейшем.

Ветвь Hotfix предназначена для исправления критических ошибок (т.е. тех, которые препятствуют работе) в версии пользователя. После исправления результат передаётся как в ветвь Master, так и в ветвь Develop.

Процесс повторяется для всех новых функциональных требований.

Команды именно по ветвлению (дублируют то, что написано в пред. билете)

git status — показывает статус текущих состояний файлов в файловой системе и информацию о ветви, в которой производится редактирование.

git log показывает журнал коммитов, а опция --graph выводит в графическом виде ветви, в которых производились данные изменения.

git diff — покажет все изменения, которые были сделаны относительно последних зафиксированных изменений.

git reset --hard HEAD сбросит все изменения, которые были сделаны в текущем локальном репозитории.

git branch — показывает ветви.

git checkout — переключает разработчика между ветвями

git merge — объединяет несколько ветвей в текущую ветвь

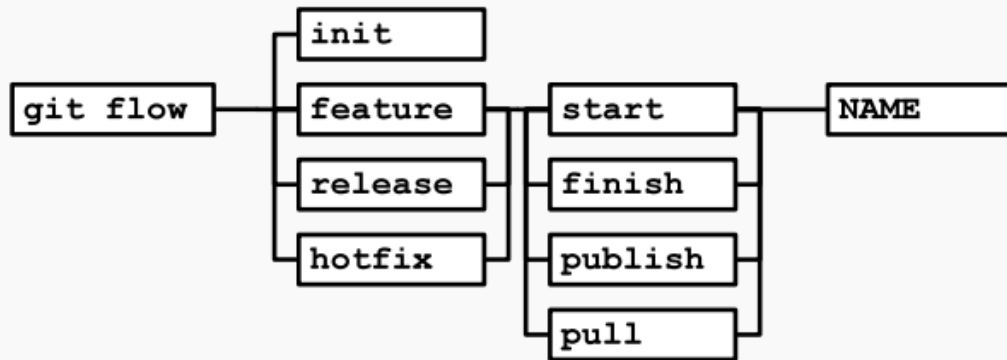
git commit — фиксирует изменения в текущей ветке. Опция -m задает сообщение коммита, которое будет показываться пользователю

git add — добавляет измененные файлы к последующему коммиту, помещая их в Stage Area.

40. GIT: Плагин git-flow.

ОПИ (МиСПО) #4. Управление изменениям, системы контроля версий (стр 86)

- Плагин для GIT — версионирование в терминах в версий, а не операций.

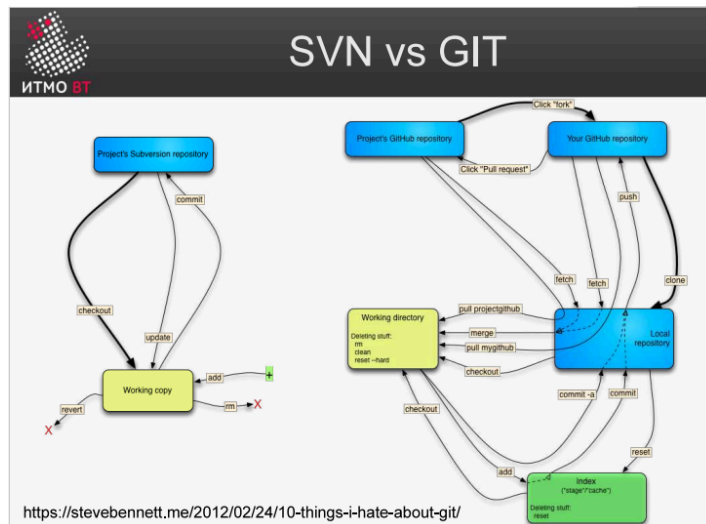


Для того, чтобы упростить работу с git(типо много команд, надо синтаксис знать), был создан плагин Git flow, позволяющий работать с версиями именно в терминах версий, а не операций, без точного знания команд и последовательностей действий.

Для использования Git flow необходима определённая начальная настройка репозитория. Для этого у Git flow есть функция init, которая при старте задаёт пользователю несколько вопросов и производит необходимую настройку.

Для создания новой feature в git flow используется команда `git flow feature start FEATURE_NAME`. Когда надо указать на окончание изменений, используется `git flow feature finish FEATURE_NAME`. Остальные команды формируются аналогичным образом. Все это позволяет пользоваться репозиторием на более высоком, независимом от синтаксиса git уровне, при этом реализуется модель, указанная на предыдущих слайдах.

Никому не нужен, хотя и упрощает обычный git



Для справки (актуально

для всех вопросов про скв)

41. Системы автоматической сборки: предпосылки появления

📺 ОПИ (МиСПО) #5 (week 6). Системы автоматической сборки (стр 89)
(завтра допишу... честно...)

42. Системы сборки: Make и Makefile.

(один билет из 3 можно и не писать. скип)
[я уже наступал на эти грабли...]

43. Системы сборки: Ant. Команды Ant.

44. Системы сборки: Ant-ivy.

45. Системы сборки: Maven. POM. Репозитории и зависимости.

46. Maven: Структура проекта. GAV.

47. Maven: Зависимости. Жизненный цикл сборки. Плагины.

48. Системы сборки: Gradle. Преимущества и файл сборки.

49. Системы сборки: GNU autotools. Создание конфигурации проекта.

50. Системы сборки: GNU autotools. Конфигурация и сборка проекта.

(ну 2 билета на сборку не попадётся... тоже скип)
[и на эти тоже наступал...]

51. Сервера сборки/непрерывной интеграции.

52. Основные понятия тестирования. Цели тестирования.

Основные термины:

- Mistake - ошибка, просчёт человека
- Fault - дефект, изъян (ПО в результате ошибки)
- Failure - неисправность, отказ, сбой (внешнее проявление дефекта)
- Error - невозможность выполнить задачу вследствие отказа
- BUG - может использоваться неформально, может обозначать: дефект, отказ, невозможность выполнить задачу...

Цели тестирования:

- Обнаружение дефектов
- Повышение уверенности в уровне качества

- Предоставление информации для принятия решений
- Предотвращение дефектов

Основная цель тестирования: увеличение приемлемого уровня пользовательского доверия в том, что программа функционирует корректно во всех необходимых обстоятельствах.

Выполняя тестирование, разработчик предлагает на основе объективных фактов (проведения набора тестов, выбора тестового покрытия для каждой части программы) поверить в то, что разработчик использовал все разумные средства, чтобы показать присутствие и исправление (а не отсутствие) дефектов.

Корректное поведение - программа должна соответствовать предъявленным к ней требованиям. Помимо требований, источником информации о корректности являются, например, особенности рынка ПО и те стандарты, которые существуют в Бизнес среде.

Пользовательское доверие должно быть заработано при помощи демонстрации разработчиком измеряемых показателей. Существуют метрики, позволяющие измерить уровень дефектов в коде (например число открытых в неделю новых дефектов, которое должно уменьшиться в процессе доработки, повышая уровень пользовательского доверия)

Важно проводить тестирование в той же среде, в которой программа будет выполняться

(Если это система управления вузом, где 5000 учащихся, должна быть проверена с одновременными запросами от 5000-10000 с учётом на рост, а не 1000 или 100 000)

53. Понятие полного тестового покрытия и его достижимости. Пример.

Понятие тестового покрытия включает в себя то, насколько код приложения покрыт тестами, которые могут находить известные и потенциальные дефекты.

Полное покрытие подразумевает покрытие тестами всего кода и всех возможных вариантов развития событий, возможных в данном коде.

Полное покрытие, если и достижимо, то ценой очень больших затрат

Пример:

`public long multiply (int A, int B)` - функция умножения двух чисел

Каждый из множителей имеет степени 2^{32} , то для полного покрытия необходимо 2^{64} операций.

Если предположить что 1 операция выполняется за 1 такт процессора, то процессор с частотой 3ГГц будет проводить тестирование в течении 181,5 лет

А в реальных программах вычислений, конечно, гораздо больше, поэтому полное покрытия является как правило не достижимым.

Самый простой способ организации тестирования - по таблице, где строка содержит оба множителя и результат - эталон. Для примера выше, хранения такой таблицы, со всеми возможными значениями и корректными результатами, является невозможным.

Важно заметить, что для построения таблицы нельзя использовать ту же самую программу, что и для самого умножения. Иначе эталонное решение будет поставлено под сомнение

54. Статическое и динамическое тестирование.

Тестирование можно выполнять ещё до того как написан код, или до того как он заработает. Для этого существуют практики статического тестирования:

Мы можем проверять наши требования, архитектуру, код в решениях аналитическим способом.

Плюс статического тестирования заключается в обнаружении дефектов ещё до начала самой разработки, или в её начале, что экономит время и средства.

Виды статического тестирования:

- Неформальное ревью
- Технический анализ (повторные просмотры)
- Management review
- Сквозной контроль
- Инспекции

Динамическое тестирование требует уже созданной программной архитектуры (осуществляется сборка и запуск модулей, групп модулей, всей системы)

Правая часть V-образной модели из валидации, то есть:

- Модульное тестирование
- Интеграционное тестирование
- Системное тестирование
- Приёмочное тестирование

Как правило динамические тесты появляются после написания кода, кроме подхода TDD (Test Driven Development), когда мы сначала продумываем тесты, и уже начинаем писать код под эти тесты. И разработка будет считаться завершённой, когда все тесты выполняются

55. Автоматизация тестов и ручное тестирование.

Вопрос, полезно ли автоматизировать тесты?

Ответ зависит от количества функциональных требований. Если продукт новый, и не устоялся на рынке, часто пренебрегают тестированием, делая упор на скорость внедрения.

Если продукт большой, долго используется с устоявшимся функционалом, то обязательно пишут автоматизированные тесты.

Но в целом, автоматизация тестов является положительным явлением, но ручное тестирование может отказаться дешевле и проще, особенно про простых задачах, где легче нанять низкоквалифицированный персонал, чем написать сложную программу для тестирования.

Отдельный вид тестирования - это регрессионное тестирование, которое заключается в том, что при изменении программы запускаются старые тесты. Это позволяет проверить, не повлияли ли изменения на работу всей программы.

Чтобы автоматизировать тесты и пользоваться регрессионным тестированием, необходимо обеспечить однозначное повторение тестового сценария

(одинаковые входные данные → одинаковые выходные данные)

Но трудно написать тесты для приемочного тестирования. Мы можем написать приёмочные испытания, но их будет делать всё равно заказчик

И автоматизация тестов не всегда уменьшает количество ручного труда. Всё зависит от размера того, сколько нужно протестировать + труд ручных тестировщиков стоит меньше чем программистов.

Также важно проверять приложение в разных окружениях (тестирование совместимости).

Из-за этого нужно выполнять много тестов, и по этому чем меньше будет сформировано функциональных тестов, тем меньшая общая сложность и длительность выполнения этих тестов

56. Источники данных для тестирования. Роли и деятельности в тестировании.

Метод чёрного ящика - у нас есть требования, спецификация, дизайн. Но внутреннее содержание программы скрыто. Тесты подают на вход программы исходные данные и сравнивают результат с известным эталоном.

Метод белого ящика - Возможно исследовать исходный код. Кроме возможности аналитически проверить корректность кода, данный метод делает возможным оценить размер тестового покрытия. Для этого из приложения строится граф, где части кода - узлы, ветвления и циклы - переходы. Затем определяется число путей обхода кода программы (цикломатическая сложность). По ним можно понять максимально необходимое число тестов, которые нужно выполнить для полной проверки программы. (таким тестированиям, например, занимаются средства разработки, статически тестируя код). *Но метод белого ящика не обязательно статическое тестирование*

Дополнительным источником данных для тестов является опыт разработчика, который позволяет решать стандартные ситуации типовыми методами, что минимизирует вероятность ошибки.

Источником данных для тестов могут являться модели, в том числе и UML-диаграммы и их описание. Из них можно выбрать основные и альтернативные пути, и протестировать их на конкретных данных

Деятельности и роли:

- Проектирование тестов

Требуют высокой квалификации, включают в себя знание предметной области, дискретной математики, особенностей пользовательского интерфейса, навыков программирования

- Автоматизация тестов

Обычно это программисты. Модульные тесты обычно пишутся создателями кода, а интеграционные - отдельно выделенными программистами с глубоким знанием архитектуры и взаимодействиях частей приложения

- Исполнение тестов

Не требуют специальной квалификации, тем не менее требует знания тестовой инфраструктуры, принцип работы с тестируемым приложением, и базовые навыки работы с компьютером

- Анализ результатов

Требуют как знания предметной области, так и технические знания. Необходима достаточно высокая квалификация

Всё это говорит о том, что для организации и проведения тестирования нужна квалифицированная подготовка. И ко многим участникам тестирования требования выше, чем к её рядовым разработчикам

57. Понятие тестового случая и сценария.

Тестовый случай состоит из набора выходных значений, предусловий выполнения, ожидаемых результатов и постусловий

Набор входных данных подразумевает определённое поведение ПО, которое определяется спецификациями требований к ПО.

Ожидаемые результаты должны создаваться как часть спецификации тестовых сценариев и включать в себя входные данные, изменения в данных и состояниях, и любые иные последствия теста.

В идеальных условиях ожидаемые результаты должны быть определены до момента выполнения теста. (для отражения этой концепции был разработан TDD)

Тестовый случай должен быть повторяемым (одинаковый ввод = одинаковый результат)

Для повторяемости, желательно автоматизировать его проведение + нужно для регрессионного тестирования

В тестовом случае должны учитываться состояния внутри ПО (если они есть) и переходы между ними.

И тестировать стоит не только нормальные пользовательские сценарии, но и сценарии, приводящие к появлению сообщений об ошибках

Тестовый сценарий - последовательность тестовых случаев

Примеры:

№	Начальное состояние	Ввод	Действие системы	Вывод	Конечное состояние
1	Готов	Пользователь вставляет карточку	Успешное чтение карточки	Приглашение "введите pin"	Ожидание pin-кода
2	Ожидание pin-кода	Вводим верный pin-код	Проверка pin-кода	Приглашение к выбору транзакции	Ожидание выбора транзакции
3	Ожидание выбора транзакции	Выбор выдачи 5000 рублей	Проверка баланса, возможности выдачи	Деньги	Выдача денег
4	Выдача денег	Пользователь берет деньги и карточку	Завершение выдачи	Благодарность за использование	Готов

№	Начальное состояние	Ввод	Действие системы	Вывод	Конечное состояние
1	Готов	Пользователь вставляет карточку	Успешное чтение карточки	Приглашение "введите pin"	Ожидание pin-кода
2	Ожидание pin-кода	Вводим неверный pin-код	Проверка pin-кода	Сообщение об ошибке	Ожидание pin-кода
3	Ожидание pin-кода	Вводим неверный pin-код	Проверка pin-кода	Сообщение об ошибке	Ожидание pin-кода
4	Ожидание pin-кода	Вводим неверный!!! pin-код	Проверка pin-кода, блокировка карточки	Сообщение об ошибке и блокировка	Готов

Важно чтобы сценарий из состояния готов, снова возвращался в состояние готов, чтобы можно было проще тестировать, при запуске тестов подряд

58. Выбор тестового покрытия и количества тестов. Анализ эквивалентности.

Требуется баланс:

Если выйти на рынок слишком рано, то мы получим негатив от пользователей.

Но с другой стороны тестировать можно очень долго и проиграть в конкуренции.

И нужно помнить, что полное тестовое покрытие недостижимо

Чтобы сократить кол-во тестов для приемлемого, нужно сократить количество тестовых случаев при помощи формальных методик:

- Эквивалентное разбиение

Производится анализ граничных значений, внутри которых тестируемая функция ведёт себя одинаково

- Таблица решений

Таблица с комбинациями входных данных (или причин) и выходных данных (либо действий)

- таблица переходов

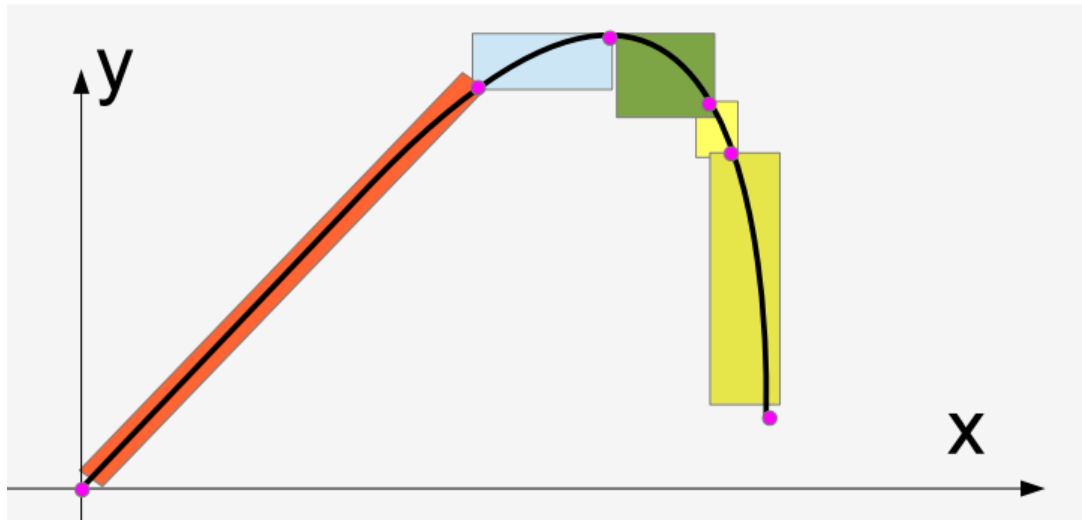
Выделяются явные состояния внутри системы, определяются переходы между этими состояниями, которые далее покрываются тестами

- Сценарии использования

Анализ эквивалентности

Тестируемая функция разбивается на участки, где программа ведёт себя одинаково. Внутри каждого участка формируется свой набор тестовых

случаев. Если таких участков не много, это позволяет сократить общее количество тестовых случаев. Отдельные тесты составляются для граничных случаев



59. Модульное тестирование. Junit 4 с примерами.

Модуль - функционально законченный элемент нашей программы. Модули описаны в дизайне

Модульное тестирование, формально, это тестирование отдельных компонентов ПО, которые изолированы от остального окружения

Модульное тестирование это первый этап тестирования после разработки (по V-образной модели)

Для модульного тестирования используются драйвера (компоненты, вызывающие модули и обеспечивающие последовательность тестирования) и заглушки (по сути замена подчинённого модуля, имеют тот же интерфейс, но с сильно более простой реализацией)

Фреймворк Junit - фреймворк для модульного тестирования, который работает на основе аннотаций. Также он является драйвером для unit тестов.

Пример программы на Junit

```
public class MyMathTest {  
  
    @Test  
    public void testAdd() {  
        MyMath math = new MyMath();  
        int result = math.add(2, 3);  
        assertEquals(5, result);  
    }  
}
```

```
}  
}
```

60. Интеграционное тестирование. Стратегии интеграции.

Компонентное интеграционное тестирование проверяет взаимодействие между программными компонентами и производится после модульного тестирования.

Смысл тестирования в проверке интерфейсов взаимодействия модулей на правильную последовательность вызовов и соответствия протоколов такого взаимодействия требованиям спецификации:

- Вызовы API, сообщения между ОО компонентами
- Базы данных пользовательский графический интерфейс
- Интерфейсы взаимодействия (сетевые, аппаратурные, локальные)
- Инфраструктурные

Интеграционное тестирование может проводиться когда 2 компонента уже разработаны. Остальные компоненты добавляются по мере их реализации. При этом важна последовательность интеграции модулей, которая определяется стратегией интеграции.

- Стратегия сверху вниз

Самая распространённая из стратегий. Применяется для бизнес приложений. Сначала бизнес логика с драйверами и заглушками, потом добавляется UI, потом блоки хранения данных

- Стратегия снизу вверх

Чаще всего используется, когда приложение сильно связано с аппаратурой. Вследствии этого интеграция начинается с блоков, тесно связанных с аппаратурой

- Функциональная End-to-End

Подразумевает постепенное наращивание приложения по функциям.

Сначала производится сборка и отладка первого пользовательского сценария "UI-Логика-БД", затем подключаются второй, третий и тд, пока приложение не будет полностью собрано

- Ядро

Сначала формируется минимальный работоспособный функционал, а потом добавляются остальные функции, расширяя возможности приложения

- Большой взрыв

Всё собирается вместе и запускается

61. Функциональное и End-to-End тестирование. Selenium.тестирование. Selenium.

Функциональное тестирование обычно рассматривается как разновидность интеграционного (или системного)

В функциональном тестировании проверяют функционал, заложенный в программу. Оно осуществляется на базе сценариев использования.

Основным элементом при функциональном тестировании является графический интерфейс.

Функциональное тестирование может проводиться как тестировщиками вручную, так и при помощи автоматических средств взаимодействия с интерфейсом

При этом полностью избежать ручного тестирования при функциональном тестировании невозможно. Особенно, если это тестирование связано с изменением функциональности (добавили поле в форму, все автоматические тесты падают)

Для функционального тестирования разработано большое число средств автоматизации, которые берут на себя управление интерфейсом пользователя, включая ввод с клавиатуры и управление мышью

Примеры:

Selenium, Sahi, Watir, Ratiional

Selenium - дополнение к браузеру Firefox. Оно позволяет сначала записать тестовую последовательность использования интерфейса, и потом сохранить её в виде тестовой программы, которая может использоваться и в других браузерах.

62. Техники статического тестирования. Статический анализ кода.

Статическое тестирование - вид тестирования ПО, который может проводиться перед динамическим тестированием. Но цель всё та же - обнаружение дефектов.

Статическое тестирование дополняет динамическое, помогая обнаружить причины сбоя(дефекты), а не сами отказы. По сути статическое тестирование происходит аналитически (например среды разработки исполняют статическое тестирование, или например Lint в c++, Java - FindBugs)

Техники:

- Неформальное ревью (рецензия)

Просите коллегу оценить принятые решения и выбранные алгоритмы в коде или спецификациях. Полезно, тк глаз может “запылиться”. Но коллега не всегда прав

- Технический анализ (повторные просмотры)

Проводится собрание под руководством технического лидера проекта, и его смотрят и проверяют по метрикам

- Management review

Собрание под руководством менеджера разработки

- Сквозной контроль

Организуется просмотр решений специально выделенным экспертом, который “ведёт” аудиторию через рассматриваемый артефакт, фиксируя недочёты и дефекты и контролирующий процесс их исправления

- Инспекции

Организованы вокруг идеи, что человек может контролировать только не более двух параметров одновременно. Вследствие этого каждый инспектор обладает двумя ролями и формально контролирует только их.

Статическое тестирование помогает:

- рано обнаруживать и исправлять дефекты
- увеличить продуктивность разработки
- уменьшить время разработки
- уменьшить время стоимости тестирования
- сокращение стоимости жизненного цикла
- уменьшение числа дефектов

Сравнение:

	Сквозной контроль	Технический Анализ	Инспекция
Основное Предназначение	Поиск дефектов	Поиск дефектов	Поиск дефектов
Дополнительная цель	Обмен опытом	Принятие решений	Улучшение процесса
Подготовка	Обычно нет	Популяризация	Формальная подготовка
Ведущий	Автор	В зависимости от обстоятельств	Подготовленный модератор
Рекомендованный размер группы	2-7	>3	3-6
Формальная процедура	Обычно нет	Иногда	Всегда
Объем материалов	небольшой	От среднего до большого	небольшой
Сбор метрик	Обычно нет	Иногда	Всегда
Выходные данные	Неформальный отчет	Формальный отчет	Список дефектов, результаты метрик, формальный отчет

63. Тестирование системы в целом. Системное тестирование. Тестирование производительности.

Тут очень мало, так что распишу в одном месте, он всё равно объединит эти вопросы в билете ↓

64. Тестирование системы в целом. Альфа- и бета тестирование. Приемочное тестирование.

После модульного и интеграционного тестирования, когда система у нас уже собрана, нам нужно проверить выполнение нефункциональных требований и показать систему пользователям

Включает в себя несколько фаз:

- Системное тестирование

Выполняется внутри компании разработчика, нужно для проверки нефункциональных требований

- Альфа тестирование

Мы берём целевую группу пользователей, зовём их внутрь организации и сажаем за подготовленные компьютеры с подготовленным окружением

- Бета тестирование

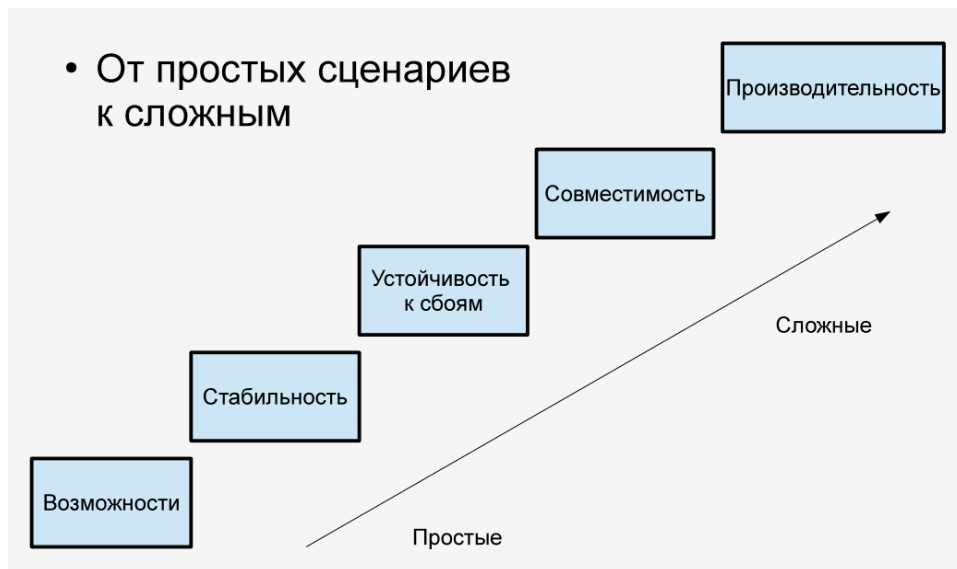
Отдаём систему заказчику, тестирование происходит в их окружении, но под контролем разработчика

- Приёмочное тестирование

Выполняется пользователями, в собственном окружении, без разработчика, в результате решается, платить или нет

Методики по сути одинаковые, только отличается строгость интерпретации результатов

Системное тестирование производится от простых сценариев к сложным



Возможности - функциональные/нефункциональные возможности, которые предоставляет наша программа

Стабильность - проверяется одновременное использование нескольких пользователей или несколько запросов от одного пользователя (пробуем вбивать реальные данные)

Устойчивость к сбоям - стараемся сломать нашу систему, кидаем неправильные данные и тп. И смотрим как наша система будет себя вести (Корректные ли сообщения об ошибках, не переходит ли система в запрещённые состояния)

Совместимость - совместимость с разными ОС, браузерами, в общем, внешнего окружения, куда может попасть наша система.

Производительность - проверяется соответствие производительности к требованиям

CARAT - Capacity, Accuracy, Response time, Availability, Throughput

Capacity - нефункциональные возможности (сколько наша система может выдержать пользователей)

Accuracy - точность вычислений, кол-во разрядов после запятой, округление и тп.

Response time - время ответа нашей системы

Availability - готовность, коэффициент, показывающий время работы нашего приложения без падения

$$Availability = \frac{MTBF - MTTR}{MTBF}, \text{ где}$$

MTBF — mean time before failure — среднее время до отказа,

MTTR — mean time to recover — среднее время до восстановления.

Throughput - Количество транзакций, данных и тп мы можем пропустить на единицу времени

Пример инструментов нагрузочного тестирования:

- *Apache Jmeter*
- *HP Load Runner*
- *The Grinder*

65. Аспекты быстродействия системы. Влияние средств измерения на результаты.

Аспекты быстродействия:

- Архитектурный аспект

Быстродействие напрямую зависит от архитектуры конкретной машины. От того из каких блоков она состоит, и как эти блоки связаны. (если программа хорошо работает на десктопе, не факт что она заработает так же хорошо на многопоточном кластере)

- Низкоуровневый аппаратный аспект

Частота процессора, количество ядер, построение кэшей и организация иерархической памяти

- Программный аспект

Используемые алгоритмы и структуры данных (пузырьковая сортировка vs quick sort). Часто более продвинутые алгоритмы занимают больше памяти

Параллелизм программы и блокировки

- Человеческий фактор

Принятие неоптимальных решений

Анализ производительности - научный эксперимент:

- Выбирается критерий оценки
- Выбираются средства измерения
 - Неинтрузивные
 - Слабоинтрузивные
 - Интрузивные

- Выбрать нагрузку/ нагрузить
- Провести анализ результатов
- Изменить один параметр/переписать один участок кода
- Повторять до полного удовлетворения

Неинтрузивные средства измерения - средства измерения, не влияющие на результаты (*бесконтактный термометр*)

Слабо интрузивные соответственно вносят небольшое искажение в работу (*градусник, который опустили в стакан воды и немного поменял её температуру*)

Интрузивные заметно искажают результаты (*Перелили воду в другой холодный стакан с датчиком температуры, из-за чего она значимо остыла*)

Если мы нашли что-то что в производительности, что нас не устраивает, мы меняем **один** параметр и повторяем процесс, пока нас всё не устроит (один потому что иначе сложно понять, что повлияло на систему)

66. Ключевые характеристики производительности.

Основные характеристики:

- Время отклика, время полного обслуживания и пропускная способность
Время отклика - время от выдачи запроса до получения первых результатов
Время полного обслуживания - время от начала запроса до его полного завершения

Пропускная способность - максимальное количество запросов за единицу времени, которое может быть обработано (пройти через канал, систему, узел)

- Утилизация (%util) и ожидание ресурса (%wait)
(%util) - показывает какую долю времени ресурс занят полезной работой
(%wait) - средняя длина очереди. показывает какой процент времени очередь не пуста

- Точка насыщения и масштабируемость
Точка насыщения - момент, когда нагрузка достигает предельного значения, которое может обработать устройство или программа (точка достижения максимально возможной производительности)
Масштабируемость - то на сколько можно количественно расширять и нагружать систему

- Эффективность

Если CPU загружен на 100% не значит что он работает эффективно

- Ускорение и прирост производительности

Смотрим после внесения изменений

- Для Java: скорость запуска и memory footprints
memory footprint - это по сути объём памяти который занимает наша программа(или отдельный объект/подсистема) во время исполнения

67. Нисходящий метод поиска узких мест.

Узкое место - это место где у нас вычисления происходят дольше всего, из-за чего другие, связанные части программы тоже начинают работать медленнее

Нисходящий метод поиска узких мест:

1. Исключить ошибки аппаратуры и администратора

Может получиться так что, например, информация с диска читается неправильно, с ошибкой и обращения к диску будут очень долгими
Нужно проверять системные журналы на наличие ошибок.

И проверять конфиги ПО, если сисадмин наладил с конфигурацией, из-за чего программе может не хватать ресурсов

2. Общесистемный и межузловой мониторинг

Смотрим на счётчики производительности ОС, получить статистику по работе CPU, памяти, ввода/вывода и тп.

3. Мониторинг приложения

Смотрим на алгоритмы, API которые мы используем.

И так же на проверяем как работает многопоточность, блокировки, синхронизация

4. Мониторинг микроархитектуры

Если всё остальное уже “вылизано”, то мы начинаем думать как уже помочь процессору. Выравнивания данных, оптимизация кэшей, борьба с пузырьками конвейера. *и это сложно...*

68. Пирамида памяти и ее влияние на производительность.

	Объем	Тд	*	Тип	Управл.
CPU	100-1000 б.	<1нс	1с	Регистр	компилятор
L1 Cache	32-128Кб	1-4нс	4с	Ассоц.	аппаратура
L2-L3 Cache	0.5-32Мб	8-20нс	30с	Ассоц.	аппаратура
Основная память	0.5Гб-4Тб	60-200нс	3-10м	Адресная	malloc, free slab-alloc
SSD	128Гб-1Тб/drive	25-250мкс	5д	Блочн.	open read/write
Жесткие диски	0.5Тб-4Тб/drive	5-20мс	4мес	Блочн.	open read/write
Магнитные ленты	1-6Тб/к	1-240с	200л	Последов.	программно

NB: при 2% cache miss производительность падает ниже 60% от идеальной!

Тут всё как обычно, быстрой памяти мало, медленной много.

Для реальных расчетов памяти процессора мало, поэтому есть кэши, которые всё ещё быстрые, но позволяют делать какие-то вычисления.

Так же обращения к соседним данным, по отношению к тем что уже лежат в кэше, тоже производится быстро, т.к кэш хранит сразу строки памяти

Мы мало как можем повлиять на аппаратуру, но мы можем подстроить наши данные, чтобы они эффективно обрабатывались, но если перейти на другой процессор, вся оптимизация скорее всего сломается

С оперативной памятью мы уже можем работать напрямую, выделять память и тп. (тот же оператор new)

69. Мониторинг производительности: процессы.

У нашего процесса есть несколько состояний:

- **Runnable** - готов к исполнению
- **On CPU** - выполняется либо на уровне системы(SYS) либо на уровне пользователя(USER)
- **Waiting for smth** - чего-то ждём.

Также у процесса есть несколько параметров:

- Квант времени (сколько этому процессу выделено времени)
- Приоритет

Пусть у нас работает процесс, его квант времени ещё не истёк, и он прерывается по clock'у (clock interrupt, то есть по времени). В этот момент ОС смотрит состояние процесса и накапливает всевозможные счётчики,

которые как раз нужны для мониторинга.

Мы сможем, например, узнать что процесс занимал время пользователя, сколько было занято времени пользователя и тп.

После этого ОС смотрит есть ли у нас процессы, готовые к исполнению с большим приоритетом (если приоритет выше текущего процесса, то наш процесс будет смещён, если нет, работаем пока не закончится квант времени)

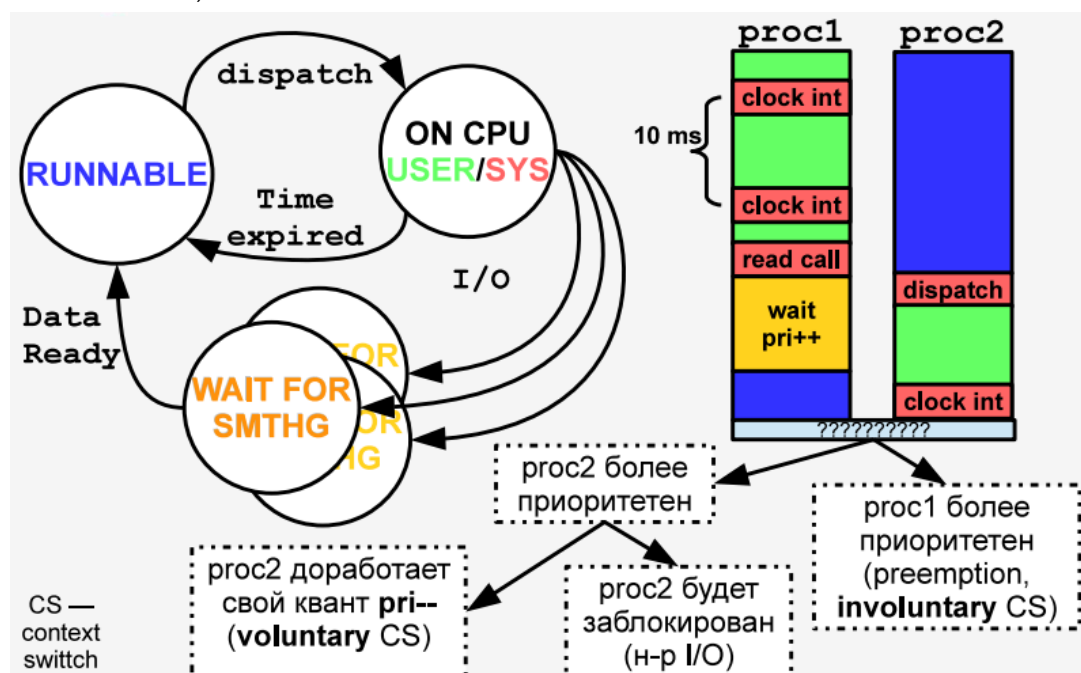
Если наша программа попадает в ввод-вывод (read call), то, чтобы не тратить время за просто так, процесс помещается в wait, а не процессор ставится другой процесс из runnable.

В этот момент так же обновляются счётчики

В момент смены процесса, все регистры смещённого процесса записываются в память и вытаскивает регистры второго процесса, и тот идёт работать

При этом, чем дольше процесс стоит в очереди тем больше у него приоритет. А если процесс доработал свой квант времени, его приоритет понижается

И в счетчиках можно узнать, сколько какой процесс, провёл в каком состоянии, сколько было смен контекста и тп



70. Мониторинг производительности: виртуальная память.

Виртуальная память - метод управления памятью компьютера, позволяющий выполнять программы, требующие больше оперативной памяти, чем

имеется в компьютере, через автоматическое перемещение частей программы между основной памятью и вторичным хранилищем. В её работе задействованы: физическая память, устройство подкачки на диск (swar-устройство), и виртуальные страницы.

Каждая страница может находиться в нескольких состояниях:

- быть в память
- быть в swar-устройстве
- быть зарезервированной

Обычно страница размером 4КБ. Размер страницы влияет на количество маппингов (запись отображений в банк памяти)

ОС её сканирует изменения страниц. Если страница давно не была изменена, то мы можем перенести её в банк памяти.

Если это сегмент кода, то мы можем просто удалить страницу, т.к у нас есть исходники, откуда был загружен этот код, и сохранять его заново бессмысленно.

Все эти механизмы запускаются когда размер свободной памяти (FREEMEM) падает до определённого размера. И чем это значение ниже, тем чаще будут происходить сканирования.

В худшем случае, запросы на выделение памяти помещаются в очередь, и ОС начинает тормозить

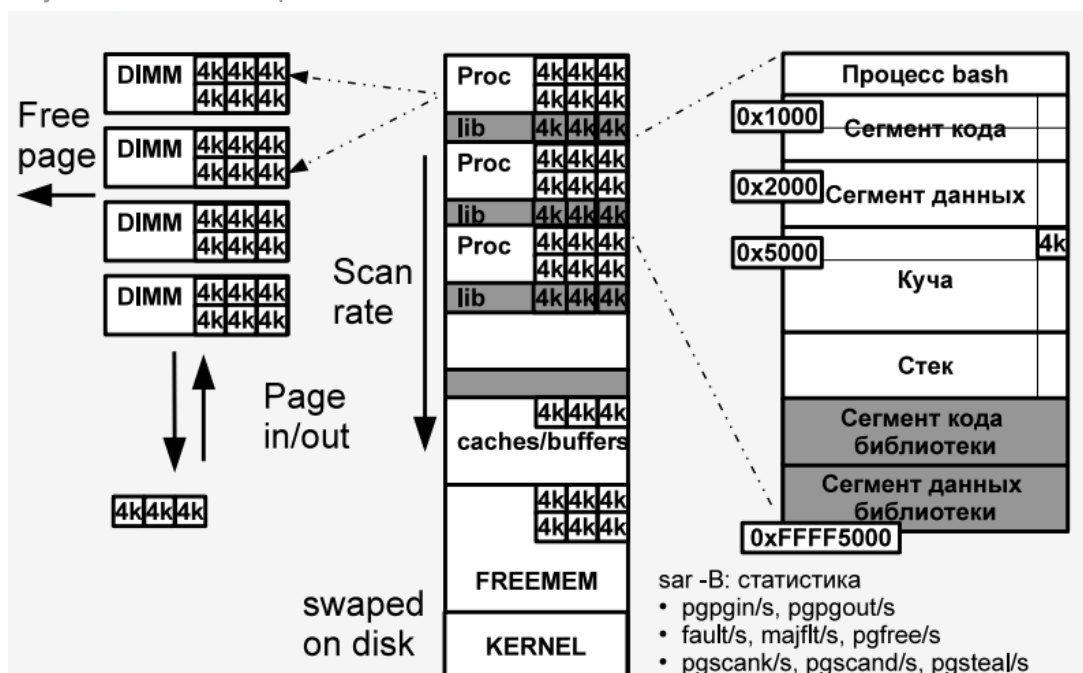
Scan rate - частота сканирований

pagein/s - сколько страничек пришло из области swar'a

pageout/s - сколько в область swar'a было записано

fault/s - ошибка сегментной адресации (не совсем ошибка, т.к страница может быть не подкачена из swar'a)

majflts/s - когда потребовалось чтение с диска



71. Мониторинг производительности: буферизированный файловый ввод-вывод.

Наш процесс, когда он выполняет операцию ввода-вывода, это сводится к тому, что выполняется библиотечная функция чтения из файла.

В операции чтения мы указываем номер файла в нашей программы, и читаем какое-то количество байт, которое должно быть перенесено в память нашей программы.

Функция делает вызов ядра ОС, и возникает программное прерывание. Ссылка на запись ввода-вывода передаётся вместе с системным вызовом и приходит к точке доступа внутри ядра (read).

Сначала нужно преобразовать имя файла в номер файла в файловой системе.

Это реализуется с помощью VFS - Virtual File System.

Буферный кэш записывает куски памяти при чтении, чтобы экономить время при многих обращениях

DNLC - это кэш поиска имён директорий/файлов в файловой системе

Далее выбирается нужное устройство для чтения (например жёсткий диск под файловой системой ext4fs)

В результате ядро формирует запрос к жёсткому диску для чтения

Далее через драйвер, мы читаем какое-то количество блоков памяти. Записываются в кэш, и нам возвращается указатель на соответствующую структуру данных.

Каждый такой запрос у нас изменяются счётчики:

r/s - чтение в секунду

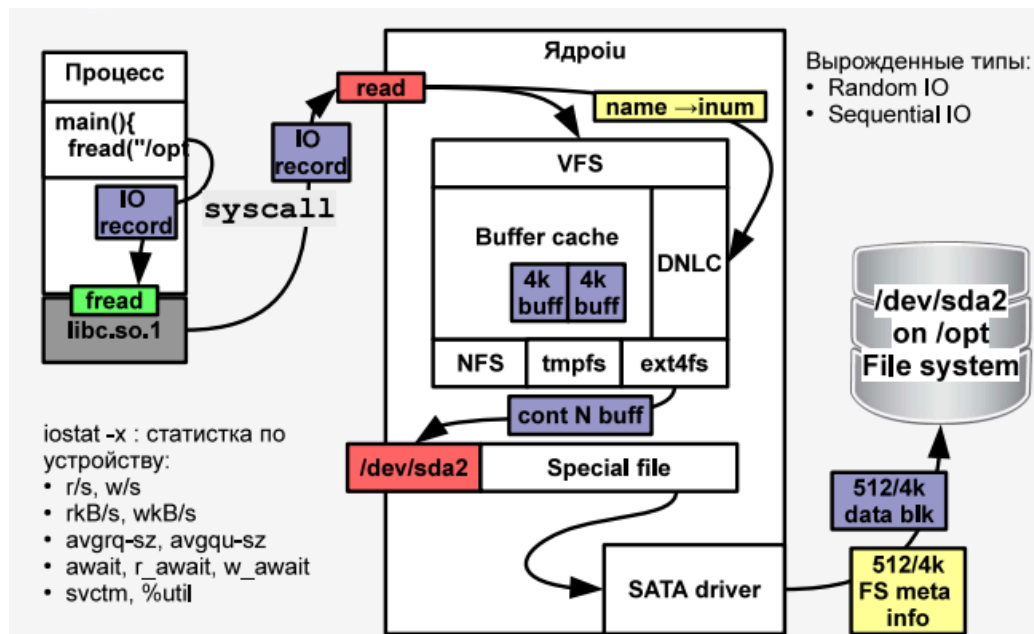
w/s - запись в секунду

rkB/s, wkB/s - чтение и запись килобайт

avgrq-sz - средний размер очереди

r_await, w_await - время ожидания

%util - общая загруженность диска



72. Мониторинг производительности: Windows и Linux.

Мониторинг Windows можно осуществлять с помощью стандартного Task Manager, или с помощью других средств, например, Resource Monitor & Performance Monitor, Reliability Monitor, Microsoft SysInternals.

Microsoft SysInternals позволяет наиболее точно посмотреть что происходит внутри операционной системы.

Они все просто собирают счётчики, но т.к их много, могут представлять разную информацию и в разном виде

Для мониторинга Linux нужно уже знать определённый набор утилит:
top - позволяет наблюдать характеристики запущенных процессов, такие как, Текущий приоритет, Занимаемая память и тп

sar - имеет множество опций для вывода системной информации ядра. Но подходы к сбору некоторых метрик устарели

perf - показывает множество характеристик ядра и запущенных под его управлением программ. Умеет работать со счётчиками производительности ядра (например, промахи кэша)

strace - утилита позволяющая делать трассировку системных вызовов, которые процесс выполняет к ядру и системным библиотекам. Достаточно интрузивен

stap - позволяет установить точки сбора информации в ядре. Использует свой собственный язык программирования.

73. Системный анализ Linux "за 60 секунд".

Эта последовательность команд поможет определить в какой подсистеме есть проблемы

- uptime - load average (кол-во процессов которые стоят в очереди ожидания процессора) за 1, 5, 15 минут
- dmesg | tail - последние ошибки
- vmstat 1 - есть ли свободная память, paging, распределение CPU
- mpstat -P ALL 1 - распределение по CPU
- pidstat 1 - статистика по процессам, горчие процессы
- iostat -xz 1 - параметры ввода-вывода
- free -m - проверка исчерпания кэшей/буферов
- sar -n DEV 1 - сетевая статистика по интерфейсам
- sar -n TCP,ETCP 1 - сетевая статистика по соединениям
- top - онлайн-мониторинг параметров

74. Создание тестовой нагрузки и нагрузчики.

Иногда запрещено наблюдать боевую систему:

- средства мониторинга вносят искажения в нормальную работу
- страх наличия дефектов в средствах мониторинга (система полностью падает если есть проблемы на уровне ядра)

Поэтому нужно сделать и нагружать тестовую систему нужно нагружать так же как и основную.

- Средства синтетической нагрузки
- средства записи реальной нагрузки

И в отличие от основной в тестовой системе можно использовать интрузивные средства мониторинга. Но синтетическая нагрузка всё равно будет сколько-то отличаться от реальной, хоть в средствах синтетической нагрузки достаточно гибкая настройка

75. Профилирование приложений. Основные подходы.

Если мы в системе ничего не нашли, нам нужно посмотреть как работает непосредственно наше приложение. Для этого используются средства и профилирования (профилировщики).

Профилировщики всегда являются интрузивными

Они нужны для анализа:

- Времени исполнения функции/метода
- Создания объектов в памяти
- Поток / соревнования за блокировку

Есть 2 основных подхода:

- Внедрение диагностических точек в указанный набор функций (AAAA! только не во все. Если профилировать всё, а в java, например гигантское кол-во методов, то программа начнёт тупить и профилирование уже будет не совсем корректным)
- Периодические прерывания с сбор информации
В этот момент времени можно просмотреть стек вызовов, и по набору этих стеков можно косвенно понять, сколько времени у нас происходит работа той или иной функции
- И если нужно рыться в микроархитектуре, то можно настроить счётчики внутри процессора

Примеры профилировщиков: Профилировщик в NetBeans, VisualVM

76. Компромиссы (trade-offs) в производительности.

Все подсистемы, алгоритмы ЭВМ связаны.

Если мы хотим, например, ускорить доступ к данным, то нам нужно больше памяти, чтобы организовать индексы и буферы (кэши)

Linear search **vs** indexing

Direct block reading **vs** buffering in memory

Выбор алгоритма и архитектуры может ускорить приложение в миллионы раз.

Та же сортировка пузырьком **vs** Qsort

сортировка пузырьком не требует много памяти для реализации, но и работает значительно медленнее

77. Рецепты повышения производительности при высоком %SYS.

Высокий %SYS - значит что время, затраченное процессором на уровне системы слишком высоко.

У этого может быть разные причины:

- Высокий ввод-вывод
- Планировщик
- Paging/swapping
- Другие системные функции ядра

При высоком вводе-выводе нужно сделать так, чтобы мы могли реже читать и писать.

Для этого можно **разбить данные на большие куски** и читать за раз большую порцию данных.

Можно **сжимать данные**, чтобы наше приложение использовало меньше информации на диске (тратим время процессора чтобы освободить дисковое время).

Также можно **заменить устройства на более быстрые**.

Можно **определить размер блока** передачи (программа должна читать данные с диска кратно размеру блока)

Если лагает планировщик: чрезмерно частая диспетчеризация, приводящая к избыточному переключению контекстов процессов и тп. В этом случае нужно проверить, не слишком ли много используется потоков, и что делает в это время ОС.

Если существуют проблемы с виртуальной памятью, и происходит избыточная подкачка страниц, можно выдать больше памяти системе за счёт сокращения памяти процессов. Другим решением может быть запрет на выгрузку из памяти определённых важных данных

Если есть какие-то другие проблемы то нужно их найти и исключить ☺
либо настроить ядро

78. Рецепты повышения производительности при высоком %IO wait.

Эти проблемы порождают высокое время ожидания CPU (высокий %IO wait)

1. Проблемы связанные с самими приложениями
Как и с высоким %SYS, следует оптимизировать запросы к диску.
реже читать-писать.

Если нужен активный обмен данными, производить его большими порциями

Проверить, согласован ли блок приложения с ОС и дисками.

(например, приложение требует блок в 16 кб, а читаем мы по 8.

Фактически в 2 раза больше обращений к памяти чем надо)

2. Буфера/кэши, т.е в системе выделено мало памяти под промежуточное хранение данных. В этом случае можно расширить память, либо настроить использование системных кэшей
3. Проблемы с аппаратурой. Можно купить новую, более совершенную дисковую подсистему, SSD, flash карты памяти.

79. Рецепты повышения производительности при высоком %Idle.

Пример: вы запустили программу, и ничего не происходит.

%User низкий, %System тоже низкий, а %Idle загружен.

Проблема - Мало RUNNABLE

Причины:

1. В системе мало процессов в стадии выполнения.

В этом случае может помочь распараллеливание алгоритмов в приложении, особенно, если система многопроцессорная. Если существуют пулы потоков-worker'ов, то в них можно добавить дополнительные потоки, распределив нагрузку по процессорам.

Ещё источником проблемы в данном случае могут быть внутренние блокировки в приложении, например, когда много потоков пытаются завладеть одним и тем же участком кода. В этом случае стоит держать блокировку как можно меньше по времени, и использовать более мягкие типы блокировок

2. Проблемы могут также быть в самой ОС
В большинстве случаев это связано с дефектами в ОС на системных блокировках. Следует посмотреть багтрекер ОС и сопоставить написанное там с симптомами конкретной проблемы.
Может также помочь настройка параметров подсистем ядра

80. Рецепты повышения производительности при высоком %User.

За тем у нас высокий %User может скрываться достаточно большое количество различных проблем. И если %User нагружен, это не значит, что наша программа хорошо работает, и что, может быть, у нас слабый процессор.

Скорее всего в программе используются какие-то не слишком подходящие алгоритмы, и где-то есть узкие места.

Чтобы найти узкое место программу нужно профилировать. Это поможет найти самые затратные функции, объекты с частым созданием/удалением.

Что с этим можно сделать:

1. Использовать алгоритмы с меньшей алгоритмической сложностью (но нужно помнить про компромиссы, если где-то прибыло, то где-то убыло)
2. Принцип повторного использования объектов. Крупные объекты и структуры нет необходимости удалять и создавать заново. Можно взять старый объект и установить в него новые данные

Такой принцип применяется в UNIX для структур процессов. Когда процесс “умирает”,

Он сначала попадает в состояние “зомби”. Когда код возврата окончательно прочитан родительским процессом, он должен окончательно “умереть” и исчезнуть из структуры ядра. Однако, ядро складывает часть “зомби” на “кладбище”, и когда необходимо создать новый процесс, он “восстает из мёртвых” - его структура повторно используется.

3. На уровне микроархитектуры важно избавляться от кэш-промахов и промахов мимо TLB (translation lookaside buffer). Кэш-промахи можно исправить работой над структурами данных (группировка данных, для попадания в одну строку кэша). Промахи мимо TLB можно исправить использованием больших страниц памяти